Distributed Compression of Latent Game Structure: A Slepian-Wolf Perspective on Multi-Agent Learning

Robert Walters

Preprint v2 | Target venue under consideration

This is a working preprint (v2). The framework is articulated, but the experimental protocol of Section 7 has not yet been executed. Tables and predictions describe *expected* outcomes; we do not report measured results. The reference implementation in Appendix B is a sketch held against the eventual run.

Abstract

Multi-agent systems often need to coordinate without reliable communication—robot teams under jamming, distributed sensors that cannot flood the network. We propose viewing such cooperation through the lens of distributed source coding: each agent's policy acts as a lossy encoder of a latent optimal action distribution, and the environment evaluates whether agents' combined outputs cohere. The intuition comes from the Slepian–Wolf theorem (1973), which showed that observers with correlated data can compress as efficiently without coordination as with it. Our setting is sequential, lossy, and non-i.i.d., so Slepian–Wolf does not apply directly; we use it as a conceptual lens, not a derivation. The lens yields four testable predictions: capacity should scale with conditional entropy $H(A_i^* \mid A_{-i}^*)$ rather than $H(A_i^*)$; agents should specialize to avoid redundant encoding; communication helps only when local uncertainty exceeds what teammates' behavior reveals; and equal-conditional-entropy agents should be permutation-invariant in expectation. We state these as conjectures, propose a corresponding information-regularized objective, and describe an experimental protocol on a small firefighting environment (*Bucket Brigade*) designed to test each prediction with bootstrap-CI estimators. Results from that protocol are deferred to a follow-up.

1 Introduction: The Coordination Paradox

The Challenge of Learning to Coordinate

A jazz ensemble improvises without sheet music or a conductor. The drummer knows when the pianist will pause. The bassist anticipates the saxophone's rhythm. No one speaks during the performance, yet coordination emerges. How?

This puzzle sits at the center of multi-agent reinforcement learning. When communication is jammed, expensive, or forbidden, agents must still learn to work together. Autonomous vehicles merging onto a highway. Distributed sensors monitoring a power grid. Robot swarms searching terrain through radio interference. In each case, agents need complementary behaviors—strategies that mesh—without any way to discuss or negotiate.

The Information-Theoretic Lens

Our perspective starts from a simple observation: learning a policy is compression. An agent distills thousands of experiences into a compact representation—its neural network weights—keeping only what matters for acting well. The question becomes: what information does each agent actually need to keep?

We view each agent's policy as a *lossy encoder*. Like a JPEG that discards fine details to shrink file size, a policy compresses the full space of possible strategies down to something manageable. It keeps the

patterns that matter and drops the rest. The environment then evaluates how well the agents' compressed strategies combine—good coordination yields high rewards, poor coordination yields low ones.

The key insight: **each agent only needs to learn the parts of the strategy that teammates cannot figure out on their own**. If Agent A can infer what Agent B will do from context, B doesn't need to encode that information explicitly. The agents can partition the strategic knowledge between them, each carrying only their share.

The Connection to Distributed Compression

This idea has a precedent. In 1973, Slepian and Wolf proved something counterintuitive about data compression: two observers with correlated data can compress just as efficiently without talking to each other as they could if they coordinated perfectly.

Picture two weather stations on opposite sides of a mountain. Both measure temperature. Their readings correlate—when it's hot on one side, it's usually hot on the other—but not perfectly. If the stations could coordinate, they'd have an obvious strategy: Station A sends its full reading, Station B sends only the difference. But what if they can't talk?

Slepian-Wolf says they can still achieve the same compression efficiency. Each station just needs to capture its *conditional information*—the part that can't be predicted from the other station's data. They don't need to agree on who sends what. They only need enough capacity to cover their own unpredictable bits.

From Compression to Coordination

Multi-agent learning has a parallel structure. Agents observe correlated slices of the same environment. Each agent's learning algorithm compresses experience into a policy. The environment then evaluates how well those policies combine—it does not literally reconstruct anything, but high rewards signal that the joint behavior preserved what mattered for the task. The analogy is conceptual rather than mathematical; we elaborate the differences below and use Slepian–Wolf as a guide for what to measure, not as a theorem we apply.

Distributed Compression	**Multi-Agent Learning**
Correlated observations	Partial views of shared environment
Compression algorithm	Learning algorithm
Compressed output	Learned policy
Reconstruction quality	Coordination quality
Compression rate	Network capacity

This analogy explains patterns that practitioners have noticed but struggled to formalize:

- **Agents specialize.** Redundant encoding wastes capacity. Agents that partition strategic knowledge outperform those that duplicate it.
- **Bigger networks plateau.** Once a network can encode its share of conditional information, extra parameters add noise, not capability.
- **Communication has a threshold.** Explicit channels help only when local uncertainty exceeds what policies can capture.

What This Paper Offers

We develop this compression-coordination connection in four directions:

1. **A unifying lens** linking distributed source coding to MARL, with random-variable definitions for how policies encode behavior and the environment evaluates joint actions.

2. **Conjectured information bounds** on decentralized coordination, expressed in conditional entropy and mutual information. We state these as conjectures, not theorems, and describe a protocol to test them.

3. **Five predictions**—capacity scaling, sample complexity, emergent specialization, communication thresholds, and symmetry invariance—each tied to a specific measurement.

4. **An information-regularized objective** using *conditional* mutual information at the representation level, designed to encourage complementary encodings without harming synchronization (a failure mode of naive MI minimization, as PMIC documented [35]).

Our intended testbed is the Bucket Brigade environment—a small firefighting task with discrete actions designed to make $H(A_i^*)$, $H(A_i^* \mid A_{-i}^*)$, and $I(\pi_i; \pi_j)$ directly estimable. Section 7 describes the protocol; the experiments themselves are deferred to follow-on work.

Why This Matters

Shannon's channel capacity theorem tells you the maximum rate at which you can send data over a noisy wire. Our framework aims for something analogous: the maximum coordination quality achievable given how much agents' observations overlap and how much their networks can hold.

The practical payoff is design guidance. Size networks based on conditional entropy, not guesswork. Use regularizers that reward complementary strategies. Know in advance which tasks will yield to decentralized learning and which will demand communication infrastructure.

The paper proceeds from background (Section 2) through the framework (Section 3), our main conjecture (Section 4), testable predictions (Section 5), algorithmic implementations (Section 6), the planned experimental protocol (Section 7, results deferred), discussion (Section 8), and conclusions (Section 9). Technical details live in the appendices. We have written for readers with background in reinforcement learning or information theory—not necessarily both.

2 Background and Related Work

This section covers the information-theoretic concepts we'll use throughout the paper. We keep it brief; fuller details are in Appendix A.

2.1 Three Concepts from Information Theory

Entropy $H(X)$ measures unpredictability. A fair coin has 1 bit of entropy. A loaded coin that always lands heads has 0 bits—no surprise. Formally:

$$H(X) = -\sum_i p_i \log_2 p_i$$

For MARL: Entropy captures how complex an agent's optimal action distribution can be. High entropy means many plausible actions; low entropy means the right choice is often obvious.

Conditional entropy $H(X|Y)$ measures what remains unpredictable about X after seeing Y. If today's weather tells you something about tomorrow's, then $H(\text{tomorrow}|\text{today}) < H(\text{tomorrow})$.

For MARL: This is the key quantity. $H(a_1^*|a_2^*)$ measures what Agent 1 needs to learn that Agent 2's behavior doesn't already reveal.

Mutual information $I(X;Y)$ quantifies overlap—how much knowing one variable tells you about the other:

$$I(X;Y) = H(X) - H(X|Y) = H(Y) - H(Y|X)$$

For MARL: High mutual information between policies means agents are encoding redundant information. Efficient coordination minimizes this redundancy.

2.2 The Slepian-Wolf Theorem

In 1973, Slepian and Wolf proved something counterintuitive: two observers with correlated data can compress just as efficiently without talking to each other as they could with perfect coordination.

Setup: Alice observes sequence X^n, Bob observes correlated sequence Y^n. Each compresses independently. A receiver gets both compressed messages and must reconstruct both sequences perfectly.

The theorem says reconstruction succeeds if and only if:

$$R_X \geq H(X|Y), \quad R_Y \geq H(Y|X), \quad R_X + R_Y \geq H(X,Y)$$

These are the same rates you'd need with full coordination. The trick: each encoder focuses on the "surprising" part of its data—the part the other observer couldn't predict. Correlation creates implicit coordination.

Important caveats for our analogy: Slepian-Wolf assumes lossless compression, i.i.d. data, and known joint distributions. Multi-agent learning violates all three: policies are lossy encoders, experience is sequential and non-stationary, and the "joint distribution" of optimal actions depends on the learned policies themselves. We use Slepian-Wolf as inspiration, not direct application.

2.3 Multi-Agent Reinforcement Learning Basics

A multi-agent system has N agents, each with partial observations and local actions. The environment transitions based on all agents' joint actions and returns rewards. Each agent learns a policy π_i mapping its observations to action distributions, typically through trial and error with only local feedback.

The core difficulty: when Agent 1 changes its policy, Agent 2's environment shifts—a moving target problem that makes learning unstable.

Training Paradigms

Independent learning ignores the problem: each agent treats others as part of the environment. Simple, but non-stationarity often hurts performance.

Centralized training, decentralized execution (CTDE) uses global information during training but deploys policies that only use local observations. VDN [29], QMIX [30], MADDPG [31], and COMA [32] follow this pattern.

Communication methods let agents exchange messages. CommNet [42], DIAL [41], and TarMAC [43] learn what to communicate; emergent communication work studies how protocols develop from scratch [45, 40].

2.4 Information Theory in RL

Several threads connect information theory to reinforcement learning, mostly in single-agent settings.

The **information bottleneck** [10] compresses representations while preserving task-relevant information: $\max I(T;Y) - \beta I(T;X)$. Applications include InfoBot [38], the variational IB [11], the conditional entropy bottleneck [12], and variational approaches in multi-agent systems [39].

Empowerment maximizes mutual information between actions and future states as intrinsic motivation [19, 20, 21].

In multi-agent settings, mutual information has been used for social influence [37], coordination [33], and exploration (MAVEN) [36].

Rate-distortion perspectives connect bounded rationality to information constraints [16, 17]—a link we develop further.

2.5 Related Frameworks

Dec-POMDPs formalize multi-agent coordination mathematically. Bernstein et al. [22] proved optimal solutions are NEXP-complete even for two agents—intractability that motivates approximations and heuristics. Oliehoek and Amato [23] provide a comprehensive treatment.

Team theory [24] studies optimal decisions with decentralized information. Witsenhausen's 1968 counterexample [25] showed that nonlinear policies can outperform linear ones—a result that still surprises.

Distributed optimization [26, 27] studies consensus without central coordination. **Mean field games** [28] handle large populations by treating other agents as a statistical distribution.

2.6 Closest Prior Work

Four lines of recent work most closely resemble what the present paper prescribes, even though none use Slepian–Wolf framing. Engaging them clarifies both what is new here and what is borrowed.

ROMA [33] learns stochastic role embeddings via two regularizers: an *identifiability* term that maximizes mutual information between an agent's role and its trajectory, and a *specialization* term that pushes apart the roles of agents with dissimilar trajectories. The effect is qualitatively what our Prediction 5.3 (emergent specialization) predicts. Our contribution relative to ROMA is the information-theoretic *account* of why such regularization works—it implements a distributed source-coding strategy—and a corresponding capacity bound.

RODE [34] extends the role idea by decomposing a multi-agent task into a small set of role-specific action spaces, with a role selector on top of role-conditioned policies. RODE empirically validates the practical value of role specialization on SMAC; we view RODE-style decomposition as an architectural realization of our conditional-entropy budget per agent.

PMIC [35] reports a finding directly relevant to our algorithmic design: simply minimizing mutual information between agents' behaviors can degrade collaboration when tasks require synchronization (e.g., simultaneous strikes). PMIC instead maximizes MI associated with high-reward joint behaviors and minimizes MI associated with low-reward ones. We take this as evidence that the redundancy penalty must be *conditioned* on task-relevant signals; Section 6 accordingly uses $I(\hat{Z}_i; \hat{Z}_j \mid R)$ or $I(\pi_i; \pi_j \mid S)$ rather than the unconditional MI.

IMAC [44] applies the information bottleneck to multi-agent communication, treating bandwidth as a capacity constraint on message entropy. Our Prediction 5.4 (communication threshold) shares the same intuition—explicit channels pay off only when local uncertainty exceeds what behavior reveals—and our Bucket Brigade communication experiment is designed to estimate the IMAC-style cost-benefit gap directly.

We also distinguish from *Coding for Distributed Multi-Agent Reinforcement Learning* [46], a recent paper whose title overlaps with ours but addresses an unrelated problem: straggler-robust coding in distributed gradient computation for MARL training, rather than the policy-as-encoder analogy.

2.7 The Gap We Address

The lines above use information-theoretic regularizers to encourage role emergence and limit communication bandwidth, but they do not unify these techniques under a single coding-theoretic frame, and they do not derive predictions about *capacity sizing* or *sample complexity* from conditional information. The present paper contributes (i) a unifying lens drawn from distributed source coding, (ii) a small set of conjectures connecting capacity, samples, specialization, communication, and symmetry to a single quantity $H(A_i^* \mid A_{-i}^*)$, and (iii) a measurement protocol designed to test all of them in one environment.

We are explicit that the connection to Slepian–Wolf is conceptual: their theorem assumes lossless coding of i.i.d. sources, while we have lossy policies acting on sequential, non-stationary data. The lens is useful because it tells us *what to measure*; it does not give us free theorems.

3 Policies as Encoders of Optimal Actions

This section formalizes our central claim: learned policies are lossy encoders of optimal behavior, and multi-agent coordination is distributed encoding.

Random Variables (reference)

- $S \sim \rho$ — state sampled from visitation distribution

- $O_i = \Omega_i(S)$ — agent i's partial observation
- $A_i^* \sim \mathcal{Z}_i(\cdot|S)$ — optimal action (from centralized teacher)
- $A_i \sim \pi_i(\cdot|O_i)$ — learned action from policy

3.1 The Encoding Intuition

A policy compresses experience into weights. Thousands of training episodes become a neural network that fits in memory. This compression is lossy—the policy forgets rare events but retains consistent patterns.

In multi-agent settings, the compression is distributed. If Robot A always takes the left corridor and Robot B always takes the right, then A doesn't need to encode knowledge about the right side. B handles that. The robots partition the strategic knowledge between them, each encoding only what the other can't infer.

3.2 The Latent Optimal Action Distribution

We posit a **latent optimal action distribution** $\mathcal{Z} : \mathcal{S} \to \Delta(\mathcal{A})$ that specifies what fully-informed, perfectly-coordinated agents would do in each state. Think of it as a centralized teacher that knows the full state and the optimal joint action.

Why a distribution rather than a deterministic function? Multiple actions may be equally good; optimal policies may maintain exploration; and computing the exact optimum may be intractable. We can formalize this with a Boltzmann distribution:

$$\mathcal{Z}(a|s) = \frac{\exp(\beta \cdot Q^*(s,a))}{\sum_{a'} \exp(\beta \cdot Q^*(s,a'))}$$

where Q^* is the optimal Q-function and β controls temperature ($\beta \to \infty$ gives deterministic greedy actions).

Each agent needs only its marginal: $\mathcal{Z}_i(a_i|s) = \sum_{a_{-i}} \mathcal{Z}(a_i, a_{-i}|s)$. This is what agent i should do, averaging over teammates' optimal behavior.

3.3 Partial Observability and the Encoding Challenge

Agents don't see the full state. Each agent i receives partial observation $O_i = \Omega_i(S)$. In a search-and-rescue task, Robot 1 sees its own position and nearby victims; Robot 2 sees different local information from its position. Neither sees the full picture.

Given only partial observations, each agent learns a policy $\pi_i : \mathcal{O}_i \to \Delta(\mathcal{A}_i)$. The policy parameters θ_i encode compressed knowledge about $\mathcal{Z}_i$. Training adjusts θ_i to make $\pi_i(\cdot|O_i)$ approximate $\mathcal{Z}_i(\cdot|S)$ as well as possible given the information bottleneck imposed by partial observability.

3.4 Key Information-Theoretic Quantities

Using the random variables defined above, we can specify the quantities that govern coordination:

Optimal action entropy $H(A_i^*)$ measures how diverse agent i's optimal behavior needs to be. A goalkeeper has low entropy (stays near goal); a midfielder has high entropy (needs varied positioning).

$$H(A_i^*) = \mathbb{E}_{S\sim\rho}\left[H(\mathcal{Z}_i(\cdot|S))\right]$$

Conditional optimal action entropy $H(A_i^*|A_{-i}^*)$ is the key quantity: the uncertainty in agent i's optimal action that can't be predicted from teammates' optimal actions. This is what agent i uniquely needs to encode.

$$H(A_i^*|A_{-i}^*) = H(A_1^*, \ldots, A_N^*) - H(A_{-i}^*)$$

If two patrol robots always take opposite sides, $H(A_1^*|A_2^*) \approx 0$—knowing one determines the other. If they patrol independently, $H(A_1^*|A_2^*) \approx H(A_1^*)$.

Policy fidelity measures how well the learned policy approximates the teacher:

$$F_{\pi_i} = \mathbb{E}_{S,O_i}\left[D_{KL}(\mathcal{Z}_i(\cdot|S)\|\pi_i(\cdot|O_i))\right]$$

Lower is better. In practice, we estimate this by sampling states, computing teacher actions, and measuring divergence from learned policy outputs.

Inter-policy redundancy $I(A_i; A_j)$ captures whether agents encode overlapping information. High redundancy can waste capacity, but it can also be necessary for synchronization; the better quantity to monitor is the *conditional* redundancy $I(A_i; A_j \mid S)$ or $I(\hat{Z}_i; \hat{Z}_j \mid R)$, which excludes the part of MI that the task itself demands [35, 12].

3.5 The Learning Cycle

Information flows through the system in a loop: nature samples a state S; agents receive partial observations O_i; policies select actions A_i; the environment transitions and returns rewards based on how well the joint actions cohere; rewards drive parameter updates that refine the encoding.

The environment is an *evaluator*, not a Shannon decoder. It does not reconstruct $\mathcal{Z}$; it scores the joint action via the reward function. The decoder language we used in Section 1 is a useful intuition pump, not a formal claim.

3.6 The Slepian-Wolf Analogy — and Its Limits

The conceptual mapping is: correlated sources $\leftrightarrow$ marginal optimal action distributions $\mathcal{Z}_1, \ldots, \mathcal{Z}_N$; encoders $\leftrightarrow$ learning algorithms producing parameters θ_i; encoding rates $\leftrightarrow$ a capacity proxy for π_i (we discuss proxies in Section 5); and *reconstruction quality* $\leftrightarrow$ coordination quality (expected return). The mapping motivates the conjecture in Section 4 but is not used to derive it.

Where the analogy is suggestive. Both settings involve (i) distributed processing of correlated information, (ii) no communication during encoding/execution, (iii) joint evaluation of combined outputs, and (iv) potential efficiency gains from exploiting correlation rather than duplicating it.

Where the analogy breaks. Slepian–Wolf assumes (a) known joint distribution, (b) lossless reconstruction, (c) i.i.d. block sources, and (d) asymptotic block length. MARL has (a) unknown, learned distributions, (b) lossy approximation by policies, (c) sequential, non-stationary data, and (d) finite samples. The Wyner–Ziv lossy extension [6] is closer in spirit but still i.i.d.; a fully appropriate formal home would be sequential rate–distortion with side information, which we do not develop here.

The upshot: we treat Slepian–Wolf as a source of *intuitions about what to measure*, not as a theorem we apply. Section 4's conjecture and Section 5's predictions are our own claims, stated as conjectures and tested empirically.

3.7 Examples

Predator-prey with specialization. Four agents hunt prey in a grid. They develop roles: chaser (high $H(A_1^*)$, needs diverse pursuit strategies), blocker (low $H(A_2^*|A_1^*)$, position largely determined by chaser), and two herders (low $I(A_3; A_4)$, complementary positioning). The team encodes $H(A_1^*, \ldots, A_4^*)$ total, distributed efficiently.

Implicit communication. Without explicit channels, agents can use early actions as signals—Agent A's opening move encodes intent, Agent B learns to decode it. This emerges naturally when it helps coordination: $I(A_A^t; A_B^{t+k}) > 0$ across time steps.

Capacity failure. Networks too small to encode $H(A_i^*|A_{-i}^*)$ bits produce policies that miss important contingencies. Coordination fails in states requiring nuanced responses. Performance plateaus below optimal—a prediction we formalize next.

4 The Distributed Learning Conjecture

Conjecture (plain language): Each agent must learn only the part of its optimal behavior that teammates can't already infer from their own observations. If every agent's policy carries at least that much unique information, the team can achieve near-optimal coordination.

4.1 The Intuition

Consider designing a rescue robot team with fixed neural network budgets. If robots develop redundant strategies—both learning to search the north side—they waste capacity and leave the south unexplored. Better: size each network based on the unique information it needs to encode, information teammates won't discover on their own.

The conjecture formalizes this. Each agent needs capacity matching its conditional optimal action entropy $H(A_i^*|A_{-i}^*)$—the uncertainty in agent i's optimal action that remains after accounting for teammates' optimal actions.

4.2 Formal Statement

We state the conjecture in approximate form first (the practical version), then note the idealized strong form. All expectations are under the stationary state distribution $\rho(S)$ induced by the learned policies or a fixed reference policy.

Conjecture 4.1 (Approximate Form): Let $A_i^* \sim \mathcal{Z}_i(\cdot|S)$ be the optimal action and $A_i \sim \pi_i(\cdot|O_i)$ the learned action. Define the information ratio $r_i = I(A_i^*; A_i|S)/H(A_i^*|A_{-i}^*)$.

For any $\epsilon > 0$, near-optimal coordination $J(\pi) \geq J^* - \epsilon$ is achievable if $r_i \geq 1 - \delta(\epsilon)$ for all agents, where $\delta \to 0$ as $\epsilon \to 0$.

Conversely, if $\min_i r_i < 1 - \Delta$ for some $\Delta > 0$, then $J(\pi) \leq J^* - \Omega(\Delta)$.

This says: when each agent's policy captures enough information about optimal actions (relative to what teammates provide), coordination succeeds; when any agent falls short, performance degrades.

Strong Form (heuristic only). A natural idealization—infinite data, universal function approximation, exact optimization, stationarity—would suggest the bound becomes tight, with optimal coordination requiring $r_i \to 1$ for all i. We do not state this as an "if and only if" theorem because the necessity direction inherits the Fano-style argument of Appendix A (Conjecture A.2), which is itself only a sketch. The idealized form is useful as a target to falsify; it is not derivable from Slepian–Wolf, which is a lossless block-coding result, not a sequential rate–distortion one.

Units and estimation: All entropies are in bits (log base 2). For discrete action spaces, estimate $H(A_i^*|A_{-i}^*)$ from teacher rollouts via plug-in estimators with Miller-Madow bias correction. Estimate $I(A_i^*; A_i|S)$ from contingency tables of (teacher action, learned action) pairs, conditioned on state. Report bootstrap confidence intervals.

4.3 Why Conditional Entropy?

The bound involves $H(A_i^*|A_{-i}^*)$, not $H(A_i^*)$. The difference matters:

- **Perfect correlation:** Two patrol robots always take opposite sides. Each has high action entropy, but $H(A_1^*|A_2^*) = 0$—knowing one determines the other. Only one needs substantial capacity.
- **Independence:** Two robots explore different floors. $H(A_1^*|A_2^*) = H(A_1^*)$—knowing the other helps nothing. Both need full capacity.
- **Partial correlation:** Two robots coordinate to lift objects. Some aspects are coupled, others independent. Each needs moderate capacity.

The Slepian-Wolf analog also has a sum constraint: $\sum_i R_{\pi_i} \geq H(A_1^*, \ldots, A_N^*)$. This global constraint (total team capacity must cover joint optimal action complexity) is usually satisfied when individual constraints are met, but becomes relevant with many agents.

4.4 Relationship to Slepian-Wolf

The conjecture preserves key Slepian-Wolf insights: conditional information is fundamental; no communication is required during encoding; efficiency comes from exploiting correlation; the environment evaluates joint outputs.

But important differences limit direct application. Slepian-Wolf assumes lossless reconstruction, known i.i.d. distributions, designed encoders, and infinite data. MARL has reward maximization, unknown non-stationary distributions, learned encoders, and finite samples. These differences explain why we state a conjecture, not a theorem, and test it empirically.

4.5 Implications

For theory: Fundamental limits exist for coordination, analogous to Shannon limits for communication. Performance should show phase transitions near the conditional entropy threshold. Environments can be characterized by their information structure, not just state/action spaces.

For practice: Size networks based on estimated conditional entropy. Design curricula starting with high-correlation tasks. Mix agent capacities based on role information requirements. Add communication when conditional entropy exceeds practical capacity.

4.6 Potential Failure Modes

The conjecture may not hold when optimal behavior requires long histories (memory architectures needed), when optimization landscapes trap learning in local optima, when observations are misaligned with task structure (requiring extra capacity for representation learning), or in adversarial settings where information hiding matters. We revisit these limitations in the Discussion.

4.7 Testable Predictions

The conjecture predicts a sigmoid-like performance curve as capacity crosses $H(A_i^*|A_{-i}^*)$; that properly regularized policies converge to information rates near the threshold; and that higher observation correlation (lower conditional entropy) improves performance at fixed capacity. We test these predictions in Section 7 using Bucket Brigade.

5 Implications and Predictions

The conjecture yields five testable predictions. For each, we state what it claims, why it matters, and how to test it in Bucket Brigade.

5.1 Prediction 1: Network Capacity

Claim: An agent's effective network capacity should match or exceed its conditional optimal action entropy. Performance improves sigmoidally as capacity crosses $H(A_i^*|A_{-i}^*)$, then plateaus.

$$\text{Effective capacity} \gtrsim H(A_i^*|A_{-i}^*)$$

Why it matters: Provides principled network sizing. Undersized networks can't encode enough strategy; oversized networks waste resources or overfit.

Caveats: "Effective capacity" has no single operational meaning. We recommend reporting multiple proxies and only drawing conclusions where they agree: (i) raw parameter count $|\theta_i|$, (ii) weight compression bits (gzip on serialized weights, or arithmetic-coding of quantized weights, as a stand-in for MDL), (iii) PAC-Bayes KL to a fixed Gaussian prior, (iv) effective-parameter count after magnitude pruning. The "bits per parameter" linkage to $H(A_i^* \mid A_{-i}^*)$ used elsewhere in this paper is a heuristic proxy, not a derivation.

Test: Sweep network sizes from $0.25\times$ to $4\times$ estimated minimum; plot performance vs capacity; look for sigmoid transition near the conditional entropy threshold.

5.2 Prediction 2: Sample Complexity

Claim (scaling hypothesis): Samples to reach ϵ-optimal performance grow *monotonically* with conditional entropy $H(A_i^* \mid A_{-i}^*)$. We do not commit to a closed form: standard PAC sample complexity involves the function-class complexity of Π_i (VC dimension, Rademacher complexity, or MDL) in addition to any target-distribution quantity, and we are not in a position to derive a tight bound across the conjunction of these factors. Schematically,

$$m_i = \tilde{O}\left(\frac{H(A_i^* \mid A_{-i}^*) + \mathcal{C}(\Pi_i)}{\epsilon^2}\right),$$

where $\mathcal{C}(\Pi_i)$ is a complexity term for the chosen policy class.

Why it matters: Even without a closed form, the qualitative prediction is useful: if conditional entropy can be *tuned down* by changing the environment (raising observation correlation), the same policy class should reach the same ϵ with strictly fewer samples.

Test: Fix the policy class. Sweep environment correlation to change $H(A_i^* \mid A_{-i}^*)$. Measure episodes to reach ϵ-optimal for several ϵ. Plot samples vs. conditional entropy; check monotonicity and report a fitted slope without claiming it is the theoretical exponent.

5.3 Prediction 3: Emergence of Specialization

Claim: Optimal policies minimize redundancy. When agents learn non-redundant encodings, they naturally specialize—spatially (different areas), temporally (different time scales), functionally (different skills), or informationally (different modalities).

Why it matters: Explains why multi-agent teams develop division of labor without explicit role assignment.

Caveats: Naively penalizing action-level MI $I(A_i; A_j)$ can hurt coordination when synchronized actions are required—a failure mode documented empirically by PMIC [35]. We instead measure redundancy at the representation level and *condition on task-relevant signal*: $I(\hat{Z}_i; \hat{Z}_j \mid R)$ during training, or $I(\pi_i; \pi_j \mid S)$ at eval; these exclude the part of MI that the task itself requires.

Test: Track pairwise policy divergence, action distribution overlap, and learned representation mutual information over training. Expect increasing divergence (specialization) until roles stabilize.

5.4 Prediction 4: Communication Thresholds

Claim: Explicit communication helps when messages M can reduce local uncertainty more than teammates' behavior already does:

$$\underbrace{H(A_i^*|O_i)}_{\text{local uncertainty}} - \underbrace{H(A_i^*|O_i, M)}_{\text{after message}} > \text{communication cost}$$

When $H(A_i^*|O_i) > H(A_i^*|A_{-i}^*)$, there's room for communication to help. When local uncertainty is already close to conditional uncertainty, adding channels provides little benefit.

Why it matters: Predicts when to invest in communication infrastructure. Useful messages include observation sharing ("I see target at X"), intention signaling ("I'll go left"), and negotiation ("you take north, I'll take south").

Test: Toggle communication bandwidth and noise; measure marginal value of additional bits; compare performance gain to conditional entropy gap.

5.5 Prediction 5: Symmetry and Invariance

Claim: When agents have equal conditional entropy—$H(A_i^*|A_{-i}^*) = H(A_j^*|A_{-j}^*)$—performance is invariant (in expectation) to agent permutation. Symmetric information requirements imply interchangeable agents.

Why it matters: Symmetric teams are robust to single-agent failure (any agent can fill any role), enable parameter sharing during training, and allow dynamic role assignment.

Caveats: Stochastic symmetry breaking during training can produce asymmetric policies even with symmetric setup. Report results across seeds; test under small asymmetries to chart robustness boundaries.

Test: Swap agent policies mid-evaluation; measure performance drop; compare to asymmetric baselines.

5.6 Summary of Predictions

Prediction	Quantity it relates	Test in Bucket Brigade	Status
P1: Capacity (§5.1)	Capacity proxy vs. $H(A_i^* \mid A_{-i}^*)$	Sweep width/depth/pruning; multiple capacity proxies	Conjecture
P2: Samples (§5.2)	Episodes to ϵ vs. $H(A_i^* \mid A_{-i}^*)$	Vary observation correlation; fit monotone trend	Scaling hypothesis
P3: Specialization (§5.3)	Conditional MI $I(\hat{Z}_i; \hat{Z}_j \mid R)$ over training	Track redundancy, role entropy, ablate λ_{red}	Conjecture
P4: Communication (§5.4)	$\Delta_i = H(A_i^* \mid O_i) - H(A_i^* \mid A_{-i}^*)$ vs. realized benefit	Toggle bandwidth B and noise; cost–benefit curve	Conjecture
P5: Symmetry (§5.5)	Permutation invariance under equal $H(A_i^* \mid A_{-i}^*)$	Swap policies mid-eval; test small asymmetries	Conjecture

All five predictions are conjectures rather than theorems, and each has a corresponding measurement protocol in Section 7. The protocol's role in this draft is to specify *what we will measure and how*, so that the eventual experiments produce falsifiable evidence rather than confirmation theater.

6 Algorithmic Framework

This section translates the theoretical framework into implementable algorithms. We propose information-regularized objectives, discuss practical estimation, and outline common pitfalls.

6.1 Information-Regularized Objective

We augment standard MARL objectives with information-theoretic regularizers, taking PMIC's caution [35] seriously: penalize *conditional* redundancy at the representation level, not unconditional MI at the action level.

$$\mathcal{L} = \mathbb{E}[R] \; - \; \lambda_{\text{red}} \sum_{i<j} I\Big(\hat{Z}_i; \hat{Z}_j \mid R\Big) \; - \; \lambda_{\text{cap}} \sum_i I\Big(\hat{Z}_i; O_i\Big)$$

where:

- **Reward maximization:** the standard RL objective.
- **Conditional redundancy penalty:** $I(\hat{Z}_i; \hat{Z}_j \mid R)$ discourages overlapping encodings *beyond* the part that the task itself induces. Conditioning on R (or, more conservatively, on S) means tasks that require synchronization are not punished for their required sync.
- **Compression penalty:** $I(\hat{Z}_i; O_i)$ in an information-bottleneck sense, keeping representations from memorizing observation noise.

Why representation-level. Action-level penalties fight tasks that require correlated outputs. Working with $\hat{Z}_i = \text{encoder}(O_i)$ lets the policy still synchronize through the action head while keeping the encoded representations distinct. This matches the architecture used by ROMA [33] and RODE [34], though our objective differs in conditioning on R.

6.2 Practical Estimation

The objective requires estimating information quantities. For discrete Bucket Brigade settings, plug-in estimators with Miller-Madow bias correction work well. For continuous settings, InfoNCE-style lower bounds with temperature tuning provide stable gradients.

Relevance: When a teacher policy $\mathcal{Z}$ is available, use cross-entropy $\text{CE}(\pi_i(\cdot|O_i), \mathcal{Z}_i(\cdot|S))$. Otherwise, use advantage-weighted log-likelihood from a centralized critic.

Redundancy: Correlation penalties between $\hat{Z}_i$ embeddings (cosine similarity, CCA, or HSIC with Gaussian kernels) are cheaper and more stable than full MI estimation.

Compression: A VIB-style KL penalty $D_{KL}(q(\hat{Z}_i|O_i)\|p(\hat{Z}_i))$ with a standard normal prior keeps representations compact.

6.3 Adaptive Capacity

The conjecture suggests sizing networks to match conditional entropy. Rather than structural changes mid-training (which can destabilize learning), we recommend:

- **Pre-training estimation:** Estimate $H(A_i^*|A^*_{-i})$ from teacher rollouts; size networks accordingly before training begins.
- **Width multipliers:** Scale hidden dimensions rather than adding/removing layers.
- **Gradual pruning:** If overparameterized, use magnitude pruning with cooldowns rather than sudden removal.
- **Hysteresis:** Require metrics to exceed thresholds for multiple epochs before triggering changes, to avoid oscillation.

6.4 Common Pitfalls

Problem	Symptom	Solution
Policy collapse	All agents learn same strategy	Increase redundancy penalty, add init noise
Underfitting	Poor performance despite convergence	Reduce compression penalty, increase capacity
MI instability	Wildly varying estimates	Use simpler surrogates, moving averages
Coordination harm	Redundancy penalty breaks sync	Penalize representation, not actions

6.5 Implementation Roadmap

1. **Baseline:** Implement standard MARL (PPO, QMIX); establish performance baseline; collect trajectories for entropy estimation.
2. **Information estimation:** Estimate conditional entropies from teacher or centralized critic; add monitoring for information metrics.
3. **Regularization:** Add redundancy penalty (start small: λ=0.01); add compression penalty if overfitting observed.
4. **Tuning:** Adjust λ values based on diagnostics; monitor specialization emergence.

6.6 Diagnostics

Track during training: per-agent reward contribution, pairwise policy divergence, representation correlation matrices, capacity utilization (via weight magnitude distributions), and information estimates (H, I) with bootstrap confidence intervals. Log (state, observation, action, teacher-action, reward) tuples for offline analysis.

Full implementation details, including distributed training and model compression for deployment, are provided in Appendix E.

7 Experimental Protocol and Expected Outcomes

Status. This section specifies the protocol we plan to run; the experiments themselves are deferred to a follow-on paper. We describe (a) the environment, (b) the quantities we will estimate, (c) the statistical protocol, and (d) the qualitative outcome each prediction would falsify. We do *not* report measured numbers.

7.1 The Bucket Brigade Environment (summary)

Bucket Brigade places N=4 agents around a ring of H=10 houses. Each turn, fires spread to neighbors with probability p_{spread} and ignite spontaneously with probability p_{ignite}. Each agent observes a local window of houses (radius r) and chooses an action from $\{\text{work\_here}, \text{work\_left}, \text{work\_right}, \text{rest}\}$. Working on a burning house extinguishes it with probability $1 - (1 - p_{\text{solo}})^{n_{\text{workers}}}$; houses burning more than τ steps are lost. Rewards include a team component for safe houses, an owner bonus, a work cost, and a rest recovery. Twelve canonical scenarios sweep the regime from trivial cooperation to high-conditional-entropy coordination. Full mechanics and scenario configurations are in Appendix C.

The design is intentionally small: N=4 agents, $|\mathcal{A}_i|$=4, $|\mathcal{O}_i|$ a low-dimensional vector. This makes oracle policies tractable to compute and the information quantities $H(A_i^*)$, $H(A_i^* \mid A_{-i}^*)$, and $I(A_i^*; A_i \mid S)$ directly estimable from rollouts.

7.2 Measurement Protocol

Information estimators. For discrete actions we use plug-in estimators with Miller–Madow bias correction [7] on held-out trajectories from a centralized teacher policy π^*. Bootstrap 95% CIs are computed by episode resampling. For sanity, we cross-check with variational lower bounds (MINE [8], InfoNCE [9]) on the same data, treating the neural estimators as offline diagnostics rather than training signal.

Capacity proxies. We report capacity via four proxies and only draw conclusions where they agree within CI:

1. parameter count $|\theta_i|$;
2. weight compression bits (gzip on serialized fp16 weights);
3. PAC-Bayes KL to a fixed isotropic Gaussian prior;
4. effective parameters after magnitude pruning (threshold tuned to retain $\geq 99\%$ training reward).

Statistical protocol. All experiments run ≥ 20 random seeds. We report mean and 95% bootstrap CI. For directional claims we report effect size (Cohen's d) and a permutation test p-value. We pre-register the conditions: sample sizes, scenario list, and seeds are fixed before runs.

7.3 Per-Prediction Protocol

P1 (Capacity, §5.1)

Manipulation. Sweep hidden-width h across five scales: $\{0.25, 0.5, 1.0, 2.0, 4.0\} \times h^*$, where h^* is the width whose four capacity proxies straddle the estimated $H(A_i^* \mid A_{-i}^*)/b$ for a fiducial b. **Outcome.** Plot team reward vs. each capacity proxy; look for a knee. *Falsifier:* if reward grows smoothly with no knee across all proxies in all scenarios, P1 is wrong.

P2 (Samples, §5.2)

Manipulation. Fix policy class; vary observation radius r to change $H(A_i^* \mid A_{-i}^*)$ (smaller $r \to$ less overlap $\to$ higher conditional entropy). For each r, measure episodes to reach reward fractions $\{0.7, 0.8, 0.9\}$ of teacher. **Outcome.** Plot $\log m_i$ vs. $\log H(A_i^* \mid A_{-i}^*)$; check monotonicity and report slope. *Falsifier:* non-monotone or inverted relationship.

P3 (Specialization, §5.3)

Manipulation. Train with the conditional redundancy penalty $\lambda_{\text{red}} I(\hat{Z}_i; \hat{Z}_j \mid R)$ at $\lambda_{\text{red}} \in \{0, 10^{-3}, 10^{-2}, 10^{-1}\}$. **Outcome.** Track $I(\hat{Z}_i; \hat{Z}_j \mid R)$, role entropy, and agent-dropout robustness over training. *Falsifier:* no monotone decrease in conditional redundancy with λ_{red}, or reward strictly worse at every $\lambda_{\text{red}} > 0$ (which would suggest the penalty just hurts).

P4 (Communication, §5.4)

Manipulation. Equip agents with a noisy cheap-talk channel of bandwidth $B \in \{0, 1, 2, 4\}$ bits and noise $\nu \in \{0, 0.1, 0.3\}$. **Outcome.** Plot realized reward gain vs. $\Delta_i = H(A_i^* \mid O_i) - H(A_i^* \mid O_i, M)$. *Falsifier:* no rank correlation between predicted gap and realized gain.

P5 (Symmetry, §5.5)

Manipulation. In a scenario where conditional entropies $H(A_i^* \mid A_{-i}^*)$ are equal by construction, permute agent policies at eval; in an asymmetric scenario do the same. **Outcome.** Symmetric: performance distribution under permutation should be statistically indistinguishable across seeds. Asymmetric: it should not. *Falsifier:* symmetric case shows large reward variance, or asymmetric case is permutation-invariant.

7.4 Baselines (planned)

Each table cell will be populated by the protocol above. We do not report numbers here; we list the comparison structure so reviewers and future runs can match it.

Baseline	Role
Independent PPO [47]	Lower bound: no information-theoretic regularization
QMIX [30]	CTDE with value factorization
MADDPG [31]	CTDE actor-critic
COMA [32]	CTDE with counterfactual baseline
ROMA [33]	MI-regularized role emergence (closest prior work)
CommNet [42] / TarMAC [43]	Explicit communication (upper bound for P4)
InfoMARL (this work)	Conditional-MI regularization, multiple capacity proxies

7.5 Reporting Commitments

When the runs complete:

- Every reward number reported with mean ± 95% CI over ≥ 20 seeds.
- Every information-theoretic quantity reported with bootstrap CI, alongside the estimator and sample count.
- Every capacity-related claim cross-checked against all four proxies; we draw a conclusion only where they agree.
- Per-prediction outcome stated as *supported / partial / falsified*, with the falsifier stated above as the test.
- Logged trajectories and configs released so the protocol can be re-run.

The point of stating the protocol now is to commit to the falsifiers *before* seeing the data.

8 Discussion

We step back from the protocol to consider what the information-theoretic lens adds to multi-agent learning, where the framework applies, what it cannot say, and what could falsify it.

8.1 What We Claim vs. What Remains Conjecture

Conceptual claims (this paper).

- Coordination without communication is usefully framed as distributed source coding.
- Conditional entropy $H(A_i^* \mid A_{-i}^*)$ is the right quantity to track—more so than $H(A_i^*)$—when reasoning about per-agent capacity needs, sample needs, and the cost of communication.
- The natural redundancy penalty for inducing specialization is *conditional* on task-relevant signal; unconditional MI penalties have documented failure modes [35].

Empirical conjectures (to be tested).

- Performance vs. capacity exhibits a knee near a conditional-entropy threshold.
- Sample complexity grows monotonically with conditional entropy at fixed policy class.
- Conditional-MI regularization induces specialization without harming synchronization.
- Communication benefit tracks $\Delta_i = H(A_i^* \mid O_i) - H(A_i^* \mid O_i, M)$.
- Permutation invariance follows from equal-conditional-entropy structure.

Out of scope. Generality of thresholds across domains, transfer to continuous control, scaling laws beyond modest N, and connections to mixture-of-experts / federated / multimodal learning are speculative; we mention them as future directions, not claims.

8.2 Broader Implications (speculative)

If the empirical conjectures hold, the framework would reframe coordination as bounded by information structure rather than by policy-class expressivity alone: networks should be "right-sized" to conditional entropy rather than scaled indefinitely, and specialization should be viewed as information-theoretic efficiency rather than an emergent surprise. Connections to mixture-of-experts (each expert encodes a partition of conditional information), multimodal learning (each modality encodes its conditional share), and federated learning (distributed encoders evaluated jointly) are worth investigating but are not supported by this paper.

8.3 Limitations

Where Slepian–Wolf breaks down. The original theorem assumes i.i.d. data, known joint distributions, lossless reconstruction, and asymptotic block length. MARL violates all of these. We treat Slepian–Wolf as conceptual inspiration; we do not claim a derivation. A more appropriate formal home would be sequential rate–distortion with side information, which we do not develop.

Challenging scenarios. Adversarial settings (conflicting objectives), highly dynamic environments (re-encoding costs dominate), communication-rich scenarios (the no-communication premise is unnecessary), and continuous control (differential entropy + estimator issues).

Missing pieces. Temporal abstraction, hierarchical decision-making, transfer learning, and causal reasoning are not addressed by this framework.

On the "this is just regularization" objection. A fair version of the objection is that ROMA, RODE, PMIC, and IMAC already practice information-theoretic regularization without Slepian–Wolf framing. Our response is that the lens provides a single quantity ($H(A_i^* \mid A_{-i}^*)$) to which capacity, samples, specialization, communication, and symmetry all reduce, plus an explicit prescription (condition on task signal) that addresses PMIC's failure mode. Whether that unification is worth its expository overhead is for the empirical work to decide.

8.4 Future Directions

Theory. Sequential rate–distortion for non-stationary sources; tighter bounds using lossy distributed coding (Wyner–Ziv); hierarchical encoding structures; formal connections to PAC-Bayes and MDL for the capacity-proxy story.

Algorithms. Learned information estimators with tighter neural bounds; information-aware architecture search; differentiable communication protocols gated by an estimate of Δ_i.

Empirics. Run the protocol of Section 7; scaling laws across N; transfer experiments measuring information overlap between tasks; robustness studies correlating specialization with agent-dropout tolerance.

9 Conclusion

This paper offers a conceptual lens, a set of conjectures, and a protocol. The lens is distributed source coding: each agent's policy compresses experience into behavior, and the environment evaluates the joint output. The conjectures connect a single quantity—conditional entropy $H(A_i^* \mid A_{-i}^*)$—to capacity requirements, sample complexity, specialization, the value of communication, and permutation invariance. The protocol commits, before any data is collected, to specific falsifiers in Bucket Brigade.

Key Takeaways

- **Policies as distributed encoders** is a productive lens; the lens is conceptual, not derivational.
- **Conditional information is the unifying quantity.** $H(A_i^* \mid A_{-i}^*)$ is what predicts capacity needs, sample needs, and the gap that communication can close.
- **The right redundancy penalty is conditional.** Penalize $I(\hat{Z}_i; \hat{Z}_j \mid R)$ or $I(\pi_i; \pi_j \mid S)$, not unconditional MI; this avoids PMIC's documented failure mode.
- **Capacity proxies must agree.** Single proxies (parameter count, gzip bits, PAC-Bayes KL, pruned effective params) each tell a partial story; conclude only where they agree.
- **Predictions, not validations.** The empirical work is future; this draft commits to the falsifiers in advance.

Open Problems

- Sequential rate–distortion bounds with side information (formal home for the conjecture).
- Robust information estimators at scale, beyond plug-in.
- Continuous control and long-horizon partial observability.
- Scaling laws across N.
- Proving (or falsifying) the distributed-learning conjecture in restricted analytical settings.

Practical Guidance for Future Runs

Log per-step tuples (S, O_i, A_i^*, A_i, R). Estimate $H(A_i^*)$, $H(A_i^* \mid A_{-i}^*)$, and $I(A_i^*; A_i \mid S)$ with plug-in + Miller–Madow, plus bootstrap CIs. Use four capacity proxies (parameter count, gzip bits, PAC-Bayes KL, pruned effective params) and report all four. Run ≥ 20 seeds; release configs and trajectories.

The pragmatic message: measure conditional information, pick a capacity proxy and stick with it, penalize *conditional* redundancy, and pre-register the falsifiers.

References

References

[1] Barlow, H.B. (1961). Possible principles underlying the transformation of sensory messages. *Sensory Communication*.

[2] Berger, T. (1971). *Rate Distortion Theory: A Mathematical Basis for Data Compression*. Prentice-Hall.

[3] Cover, T.M. & Thomas, J.A. (2006). *Elements of Information Theory* (2nd ed.). Wiley-Interscience.

[4] Csiszár, I. & Körner, J. (2011). *Information Theory: Coding Theorems for Discrete Memoryless Systems* (2nd ed.). Cambridge University Press.

[5] Slepian, D. & Wolf, J. (1973). Noiseless coding of correlated information sources. *IEEE Trans. Information Theory*, 19(4):471–480.

[6] Wyner, A.D. & Ziv, J. (1976). The rate-distortion function for source coding with side information at the decoder. *IEEE Trans. Information Theory*, 22(1):1–10.

[7] Miller, G.A. (1955). Note on the bias of information estimates. In *Information Theory in Psychology*.

[8] Belghazi, M.I., Baratin, A., Rajeshwar, S., Ozair, S., Bengio, Y., Courville, A., & Hjelm, R.D. (2018). MINE: Mutual information neural estimation. *ICML*.

[9] van den Oord, A., Li, Y., & Vinyals, O. (2018). Representation learning with contrastive predictive coding. *arXiv:1807.03748*.

[10] Tishby, N. & Zaslavsky, N. (2015). Deep learning and the information bottleneck principle. *IEEE Information Theory Workshop*.

[11] Alemi, A.A., Fischer, I., Dillon, J.V., & Murphy, K. (2017). Deep variational information bottleneck. *ICLR*.

[12] Fischer, I. (2020). The conditional entropy bottleneck. *Entropy*, 22(9):999.

[13] Strouse, D. & Schwab, D.J. (2017). The deterministic information bottleneck. *Neural Computation*.

[14] Tishby, N. & Polani, D. (2011). Information theory of decisions and actions. In *Perception–Action Cycle*, Springer.

[15] Zbontar, J., Jing, L., Misra, I., LeCun, Y., & Deny, S. (2021). Barlow Twins: Self-supervised learning via redundancy reduction. *ICML*.

[16] Ortega, P.A. & Braun, D.A. (2013). Thermodynamics as a theory of decision-making with information-processing costs. *Proc. Royal Society A*, 469(2153).

[17] Genewein, T., Leibfried, F., Grau-Moya, J., & Braun, D.A. (2015). Bounded rationality, abstraction, and hierarchical decision-making. *Frontiers in Robotics and AI*, 2:27.

[18] Grau-Moya, J., Leibfried, F., & Bou-Ammar, H. (2018). Balancing two-player stochastic games with soft Q-learning. *IJCAI*.

[19] Klyubin, A.S., Polani, D., & Nehaniv, C.L. (2005). Empowerment: A universal agent-centric measure of control. *IEEE CEC*.

[20] Mohamed, S. & Rezende, D.J. (2015). Variational information maximisation for intrinsically motivated reinforcement learning. *NeurIPS*.

[21] Eysenbach, B., Gupta, A., Ibarz, J., & Levine, S. (2018). Diversity is all you need: Learning skills without a reward function. *ICLR 2019*; arXiv:1802.06070.

[22] Bernstein, D.S., Givan, R., Immerman, N., & Zilberstein, S. (2002). The complexity of decentralized control of Markov decision processes. *Mathematics of Operations Research*, 27(4):819–840.

[23] Oliehoek, F.A. & Amato, C. (2016). *A Concise Introduction to Decentralized POMDPs*. Springer.

[24] Marschak, J. & Radner, R. (1972). *Economic Theory of Teams*. Yale University Press.

[25] Witsenhausen, H.S. (1968). A counterexample in stochastic optimum control. *SIAM J. Control*, 6(1):131–147.

[26] Boyd, S., Parikh, N., Chu, E., Peleato, B., & Eckstein, J. (2011). Distributed optimization and statistical learning via ADMM. *Foundations and Trends in Machine Learning*, 3(1):1–122.

[27] Nedić, A. & Ozdaglar, A. (2009). Distributed subgradient methods for multi-agent optimization. *IEEE Trans. Automatic Control*, 54(1):48–61.

[28] Huang, M., Caines, P.E., & Malhamé, R.P. (2006). Large-population cost-coupled LQG problems with nonuniform agents. *IEEE Trans. Automatic Control*.

[29] Sunehag, P., et al. (2018). Value-decomposition networks for cooperative multi-agent learning. *AAMAS*.

[30] Rashid, T., Samvelyan, M., Schroeder de Witt, C., Farquhar, G., Foerster, J., & Whiteson, S. (2018). QMIX: Monotonic value function factorisation for deep multi-agent reinforcement learning. *ICML*.

[31] Lowe, R., Wu, Y., Tamar, A., Harb, J., Abbeel, P., & Mordatch, I. (2017). Multi-agent actor-critic for mixed cooperative-competitive environments. *NeurIPS*.

[32] Foerster, J., Farquhar, G., Afouras, T., Nardelli, N., & Whiteson, S. (2018). Counterfactual multi-agent policy gradients (COMA). *AAAI*.

[33] Wang, T., Dong, H., Lesser, V., & Zhang, C. (2020). ROMA: Multi-agent reinforcement learning with emergent roles. *ICML*.

[34] Wang, T., Gupta, T., Mahajan, A., Peng, B., Whiteson, S., & Zhang, C. (2021). RODE: Learning roles to decompose multi-agent tasks. *ICLR*.

[35] Li, P., Tang, H., Yang, T., Hao, X., Sang, T., Zheng, Y., Hao, J., Taylor, M.E., Tao, W., & Wang, Z. (2022). PMIC: Improving multi-agent reinforcement learning with progressive mutual information collaboration. *ICML*.

[36] Mahajan, A., Rashid, T., Samvelyan, M., & Whiteson, S. (2019). MAVEN: Multi-agent variational exploration. *NeurIPS*.

[37] Jaques, N., et al. (2019). Social influence as intrinsic motivation for multi-agent deep reinforcement learning. *ICML*.

[38] Goyal, A., Islam, R., Strouse, D., Ahmed, Z., Larochelle, H., Botvinick, M., Bengio, Y., & Levine, S. (2019). InfoBot: Transfer and exploration via the information bottleneck. *ICLR*.

[39] Igl, M., Gambardella, A., He, J., Nardelli, N., Siddharth, N., Boehmer, W., & Whiteson, S. (2019). Multitask soft option learning. *UAI 2020*; arXiv:1904.01033.

[40] Eccles, T., Bachrach, Y., Lever, G., Lazaridou, A., & Graepel, T. (2019). Biases for emergent communication in multi-agent reinforcement learning. *NeurIPS*.

[41] Foerster, J., Assael, Y.M., de Freitas, N., & Whiteson, S. (2016). Learning to communicate with deep multi-agent reinforcement learning (DIAL). *NeurIPS*.

[42] Sukhbaatar, S., Szlam, A., & Fergus, R. (2016). Learning multiagent communication with backpropagation (CommNet). *NeurIPS*.

[43] Das, A., Gervet, T., Romoff, J., Batra, D., Parikh, D., Rabbat, M., & Pineau, J. (2019). TarMAC: Targeted multi-agent communication. *ICML*.

[44] Wang, R., He, X., Yu, R., Qiu, W., An, B., & Rabinovich, Z. (2020). Learning efficient multi-agent communication: An information bottleneck approach. *ICML*.

[45] Lazaridou, A., Peysakhovich, A., & Baroni, M. (2017). Multi-agent cooperation and the emergence of (natural) language. *ICLR*.

[46] Wang, B., Xie, J., & Atanasov, N. (2021). Coding for distributed multi-agent reinforcement learning. *ICRA*; arXiv:2101.02308. (Distinct from this paper: addresses straggler coding in distributed training, not policy-as-encoder.)

[47] Schulman, J., Wolski, F., Dhariwal, P., Radford, A., & Klimov, O. (2017). Proximal policy optimization algorithms. *arXiv:1707.06347*.

[48] Saxe, A.M., McClelland, J.L., & Ganguli, S. (2019). A mathematical theory of semantic development in deep neural networks. *PNAS*, 116(23):11537–11546.

A Mathematical Details

This appendix collects the formal definitions and supporting arguments for the conjectures in the main text. *We have downgraded several v1 "theorems" to conjectures or heuristics where the proof was a sketch rather than a derivation*, since the v1 review correctly noted that load-bearing Fano- and covering-number-based arguments were too thin to label as theorems. The conjectures here are what we test in Section 7; the definitions are unchanged.

A.1 Foundational Definitions

A.1.1 Multi-Agent Systems

Definition A.1 (Multi-Agent Markov Decision Process): A Multi-Agent MDP (MAMDP) is a tuple $\mathcal{M} = (\mathcal{S}, \mathcal{O}, \mathcal{A}, P, R, \gamma, \rho_0, N)$ where:

- $\mathcal{S}$ is the state space (possibly infinite)
- $\mathcal{O} = \mathcal{O}_1 \times ... \times \mathcal{O}_N$ is the joint observation space
- $\mathcal{A} = \mathcal{A}_1 \times ... \times \mathcal{A}_N$ is the joint action space
- $P : \mathcal{S} \times \mathcal{A} \to \Delta(\mathcal{S})$ is the state transition function
- $R : \mathcal{S} \times \mathcal{A} \to \mathbb{R}^N$ is the reward function
- $\gamma \in [0, 1)$ is the discount factor
- $\rho_0 \in \Delta(\mathcal{S})$ is the initial state distribution
- $N \in \mathbb{N}$ is the number of agents

For each agent $i \in \{1, ..., N\}$:

- $\Omega_i : \mathcal{S} \to \mathcal{O}_i$ is the observation function
- $\pi_i : \mathcal{O}_i \to \Delta(\mathcal{A}_i)$ is the policy

Definition A.2 (Observation Correlation): The correlation between agents' observations is characterized by the mutual information:

$$\mathcal{C}_{ij} = I(O_i; O_j) = \mathbb{E}_{s \sim \rho}\left[\log \frac{P(o_i, o_j|s)}{P(o_i|s)P(o_j|s)}\right]$$

where ρ is the stationary state distribution induced by the joint policy $\pi = (\pi_1, ..., \pi_N)$.

Definition A.3 (Partial Observability Degree): The degree of partial observability for agent i is:

$$\mathcal{P}_i = 1 - \frac{I(S; O_i)}{H(S)} \in [0, 1]$$

where:

- $\mathcal{P}_i = 0$ means full observability (agent sees entire state)
- $\mathcal{P}_i = 1$ means no observability (observations are independent of state)
- Intermediate values quantify the degree of information loss

A.1.2 Information-Theoretic Quantities

Definition A.4 (Entropy for Continuous and Discrete Variables): For a discrete random variable X with probability mass function $p(x)$:

$$H(X) = -\sum_{x \in \mathcal{X}} p(x) \log p(x)$$

For a continuous random variable X with probability density function $f(x)$:

$$h(X) = -\int_{\mathcal{X}} f(x) \log f(x) dx$$

Note: We use H for discrete entropy and h for differential entropy.

Definition A.5 (Conditional Entropy): For discrete random variables X and Y:

$$H(X|Y) = \sum_{y \in \mathcal{Y}} p(y) H(X|Y = y) = H(X, Y) - H(Y)$$

This measures the average uncertainty in X given knowledge of Y.

Definition A.6 (Mutual Information): The mutual information between X and Y is:

$$I(X; Y) = H(X) - H(X|Y) = H(Y) - H(Y|X) = H(X) + H(Y) - H(X, Y)$$

This quantifies the reduction in uncertainty about one variable given knowledge of the other.

Definition A.7 (Kullback-Leibler Divergence): For distributions P and Q over the same space:

$$D_{KL}(P \| Q) = \mathbb{E}_{x \sim P} \left[\log \frac{P(x)}{Q(x)} \right]$$

This measures the "distance" from distribution Q to distribution P.

A.2 The Latent Optimal Action Distribution

A.2.1 Formal Construction

Definition A.8 (Optimal Q-Function): The optimal Q-function for the joint system is:

$$Q^*(s, a) = R(s, a) + \gamma \mathbb{E}_{s' \sim P(\cdot|s,a)} \left[\max_{a' \in \mathcal{A}} Q^*(s', a') \right]$$

Definition A.9 (Latent Optimal Action Distribution): The latent optimal action distribution is defined via the softmax (Boltzmann) distribution:

$$\mathcal{Z}(a|s) = \frac{\exp(\beta Q^*(s,a))}{\sum_{a' \in \mathcal{A}} \exp(\beta Q^*(s,a'))}$$

where $\beta \in [0, \infty)$ is the optimality temperature:

- $\beta \to 0$: Uniform distribution (maximum entropy)
- $\beta \to \infty$: Deterministic optimal action (minimum entropy)
- Finite β: Balances optimality and exploration

Proposition A.1 (Properties of $\mathcal{Z}$): The latent optimal action distribution satisfies:

1. **Normalization**: $\sum_{a \in \mathcal{A}} \mathcal{Z}(a|s) = 1$ for all $s \in \mathcal{S}$
2. **Optimality**: $\mathbb{E}_{a \sim \mathcal{Z}(\cdot|s)}[Q^*(s,a)]$ is maximized among all distributions
3. **Smooth approximation**: As $\beta \to \infty$, $\mathcal{Z}$ converges to the deterministic optimal policy

Proof:

1. Follows from the definition of softmax
2. The softmax distribution maximizes $\mathbb{E}[Q] - \frac{1}{\beta} H(\pi)$, which approaches $\max_a Q^*(s,a)$ as $\beta \to \infty$
3. By properties of the softmax function ■

A.2.2 Marginalization and Conditioning

Definition A.10 (Marginal Optimal Action Distribution): For agent i, the marginal distribution is:

$$\mathcal{Z}_i(a_i|s) = \sum_{a_{-i} \in \mathcal{A}_{-i}} \mathcal{Z}(a_i, a_{-i}|s)$$

where $a_{-i} = (a_1, ..., a_{i-1}, a_{i+1}, ..., a_N)$ denotes all actions except agent i's.

Lemma A.1 (Entropy Decomposition): The joint entropy decomposes as:

$$H(\mathcal{Z}_1, ..., \mathcal{Z}_N) = \sum_{i=1}^{N} H(\mathcal{Z}_i | \mathcal{Z}_1, ..., \mathcal{Z}_{i-1})$$

Proof: By chain rule of entropy:

$$H(X_1, ..., X_N) = H(X_1) + H(X_2|X_1) + ... + H(X_N|X_1, ..., X_{N-1})$$

Apply with $X_i = \mathcal{Z}_i$ ■

A.3 The Policy Encoding Framework

A.3.1 Policies as Encoders

Definition A.11 (Policy as Encoder): A policy $\pi_i : \mathcal{O}_i \to \Delta(\mathcal{A}_i)$ with parameters θ_i acts as an encoder:

- **Input**: Observation $o_i = \Omega_i(s)$
- **Encoding function**: $f_i(o_i; \theta_i) = \pi_i(\cdot|o_i)$
- **Encoded message**: Action distribution over $\mathcal{A}_i$
- **Reconstruction target**: $\mathcal{Z}_i(\cdot|s)$

Definition A.12 (Policy Fidelity): The fidelity of policy π_i to optimal actions is measured by mutual information:

$$I(A_i^*; A_i|S) = H(A_i^*|S) - H(A_i^*|A_i, S)$$

where $A_i^* \sim \mathcal{Z}_i(\cdot|S)$ and $A_i \sim \pi_i(\cdot|O_i)$.

Note: This differs from KL divergence; MI is symmetric and measures statistical dependence.

Heuristic A.1 (Capacity Proxy). For any policy π_i with parameters $\theta_i \in \mathbb{R}^m$ stored at finite precision b bits per coordinate, an information-rate proxy is

$$R_{\pi_i} \lesssim \alpha \cdot b \cdot m,$$

where $\alpha \in (0, 1]$ is an architecture-dependent "efficiency" factor that we do not derive in closed form. The argument is informal: under a fixed quantization scheme, $H(\theta_i) \leq bm$, and by data processing $I(\theta_i; A_i^*) \leq H(\theta_i)$; α absorbs the gap between $H(\theta_i)$ and the information actually *usable* for predicting A_i^*. We use this only as a proxy and recommend reporting it alongside three alternatives: gzip-compression bits of serialized weights (an MDL stand-in), PAC-Bayes KL to a fixed prior, and effective-parameter count after magnitude pruning. Conclusions should require all four to agree within CI.

A.4 The Distributed Learning Bounds

A.4.1 Necessity of Conditional Information

Conjecture A.2 (Lower Bound on Information Rate). For any set of policies $\{\pi_i\}_{i=1}^N$ achieving $J(\pi) \geq J^* - \epsilon$, we conjecture

$$I(A_i^*; A_i \mid S) \;\geq\; H(A_i^* \mid A_{-i}^*) - \delta(\epsilon),$$

with $\delta(\epsilon) \to 0$ as $\epsilon \to 0$.

Heuristic argument (not a proof). Consider a teacher–student setup in which each agent i "decodes" A_i^* from A_i and access to teammate actions A_{-i}^*. A Lipschitz assumption on J in policy KL,

$$J^* - J(\pi) \;\leq\; L \cdot \sum_i \mathbb{E}_S\big[D_{\mathrm{KL}}\big(\mathcal{Z}_i(\cdot \mid S) \,\|\, \pi_i(\cdot \mid O_i)\big)\big]\,,$$

turns a performance gap ϵ into a KL budget, and Pinsker's inequality + a standard Fano-style argument over the action alphabet $\mathcal{A}_i$ then bounds $H(A_i^* \mid A_i, A_{-i}^*)$ from above and hence $I(A_i^*; A_i \mid S)$ from below by $H(A_i^* \mid A_{-i}^*) - \delta(\epsilon)$. The catch is that the Lipschitz assumption is non-trivial—it requires the advantage function to be smooth in policy KL, which is not in general true for non-myopic MDPs. We therefore state this as a conjecture, not a theorem.

Why we removed v1's proof. The v1 sketch wrote $P(\pi_i \neq \mathcal{Z}_i \mid \mathcal{Z}_{-i})$, but π_i and $\mathcal{Z}_i$ are conditional distributions; the equality is type-incorrect. A clean Fano application would require a discrete reconstruction event (e.g., MAP-decoded action $\hat{A}_i$ vs. A_i^*), which then needs the Lipschitz-in-KL assumption above to connect reconstruction error to ϵ. The v1 "proof" folded both steps together implicitly and could not be made rigorous as written; we prefer to make the dependency on Lipschitz-in-KL explicit and downgrade to conjecture.

A.4.2 Sufficiency of Conditional Information

Theorem A.3 (Achievability): There exist policies $\{\pi_i\}_{i=1}^N$ with:

$$R_{\pi_i} = H(\mathcal{Z}_i|\mathcal{Z}_{-i}) + o(1)$$

achieving $J(\pi) \to J^*$ as the number of samples $n \to \infty$.

Proof Outline:

1. Construct policies using maximum likelihood estimation from samples
2. Apply universal source coding theorems
3. Show convergence using concentration inequalities

(Full proof requires measure-theoretic arguments beyond our scope)

A.5 Sample Complexity Analysis

A.5.1 Information-Theoretic Sample Complexity

Scaling Hypothesis A.4 (Sample Complexity). For a fixed policy class Π_i with complexity measure $\mathcal{C}(\Pi_i)$ (VC dimension, Rademacher complexity, or MDL), we conjecture that the number of samples needed to reach ϵ-optimal performance with probability $1 - \delta$ is

$$m_i \;=\; \tilde{O}\Big(\frac{H(A_i^* \mid A_{-i}^*) + \mathcal{C}(\Pi_i) + \log(1/\delta)}{\epsilon^2}\Big).$$

The H term reflects the difficulty of the target distribution; the $\mathcal{C}$ term reflects the difficulty of the function class learning it.

Why we removed v1's closed form. The v1 "proof" asserted a covering-number bound $\mathcal{N}(\epsilon, \Pi_i) \leq \exp(H(A_i^* \mid A_{-i}^*)/\epsilon)$ without justification. Standard PAC covering bounds for policy classes are in terms of class complexity (VC, Rademacher, MDL), not the entropy of the *target* distribution. A correct version of the bound has *both* terms above, and we are not in a position to derive their interaction in closed form across realistic policy classes. We therefore state the relationship as a monotone scaling hypothesis to be tested rather than a theorem.

A.6 Network Capacity Analysis

A.6.1 Capacity of Neural Networks

Definition A.13 (Effective Capacity): For a neural network with architecture $\mathcal{N}$ and parameters $\theta \in \mathbb{R}^m$:

$$C_{\text{eff}}(\mathcal{N}) = \alpha(\mathcal{N}) \cdot b \cdot m$$

where $\alpha(\mathcal{N}) \in (0, 1]$ depends on:

- Network depth L
- Activation functions
- Connectivity pattern

Architecture efficiency (omitted). The v1 draft asserted closed-form values of α for fully-connected, convolutional, and attention architectures ($\alpha \approx 1/L$, k^2/s^2, $1/\sqrt{d}$ respectively), citing dynamics-of-learning analyses [48]. We have removed these formulas because they were not derivable from the cited work and were treated as theorems. Architecture-specific efficiency is an empirical question; we instead recommend measuring the four capacity proxies in Section 7 and treating their cross-architecture ranking as the empirical statement.

A.6.2 Minimum Capacity Theorem

Proxy A.6 (Capacity Threshold Heuristic). Combining Heuristic A.1 with Conjecture A.2 *at the proxy level* gives the schematic threshold

$$|\theta_i|_{\min} \;\approx\; \frac{H(A_i^* \mid A_{-i}^*)}{\alpha \cdot b},$$

which we use only as a sizing prior in Section 7's capacity sweep (it sets the central width h^* around which we sweep $\{0.25, 0.5, 1, 2, 4\}\times$). The threshold inherits the heuristic status of both inputs; the experiment tests *whether a knee exists near the proxy-implied value*, not whether the proxy is correct.

A.7 Specialization and Redundancy

A.7.1 Redundancy Minimization

Definition A.14 (Inter-Policy Redundancy): The redundancy between policies π_i and π_j is:

$$\mathcal{R}_{ij} = I(\pi_i; \pi_j) = H(\pi_i) + H(\pi_j) - H(\pi_i, \pi_j)$$

Proposition A.7 (Lagrangian framing). When per-agent capacity $|\theta_i|$ is constrained, the Lagrangian relaxation of the constrained problem has the form

$$\min_{\pi_1,\ldots,\pi_N} \left[-J(\pi) + \lambda \sum_{i<j} \mathcal{R}_{ij} \right]$$

for some $\lambda > 0$ tied to the capacity multipliers. This is a structural observation, not a claim about *unconditional* optimality: removing the capacity constraint and the Lagrangian, naive redundancy minimization can hurt collaboration in tasks requiring synchrony, as PMIC [35] demonstrates. The redundancy term in Section 6 accordingly uses conditional MI.

A.7.2 Emergence of Roles

Definition A.15 (Role Clarity): The role clarity of agent i is:

$$\text{Clarity}_i = \frac{H(\mathcal{Z}_i) - H(\mathcal{Z}_i|\mathcal{Z}_{-i})}{H(\mathcal{Z}_i)} = \frac{I(\mathcal{Z}_i; \mathcal{Z}_{-i})}{H(\mathcal{Z}_i)}$$

This measures how much of agent i's behavior is determined by others' strategies.

Proposition A.2 (Specialization Metrics): High specialization corresponds to:

1. Low redundancy: $\mathcal{R}_{ij} \to 0$ for all $i \neq j$
2. High role clarity: $\text{Clarity}_i \to 1$
3. Efficient encoding: $R_{\pi_i} \approx H(\mathcal{Z}_i|\mathcal{Z}_{-i})$

A.8 Communication Value Analysis

A.8.1 When Communication Helps

Definition A.16 (Communication Value): The value of communication channel with bandwidth B bits is:

$$V_{\text{comm}}(B) = J(\pi_B^{\text{comm}}) - J(\pi^{\text{no-comm}})$$

where π_B^{comm} allows B bits of communication per timestep.

Conjecture A.8 (Communication Threshold). A necessary condition for some message M to yield positive communication value is

$$\sum_{i=1}^{N} H(A_i^* \mid O_i) \;>\; \sum_{i=1}^{N} H(A_i^* \mid A_{-i}^*).$$

Intuition. Without communication, agent i must compress $H(A_i^* \mid O_i)$ bits of uncertainty using whatever its policy can encode; with perfect coordination among teammates, only $H(A_i^* \mid A_{-i}^*)$ bits remain. Communication can close this gap if a message M can reduce $H(A_i^* \mid O_i)$ toward $H(A_i^* \mid A_{-i}^*)$. The condition is necessary, not sufficient: realizing the gap requires that the channel actually carry enough mutual information with A_{-i}^*, and that the cost of the channel be below the realized benefit. Bandwidth B does not appear in the inequality; it enters through the maximum reducible KL on the right side, which is bounded by B.

A.9 Convergence Analysis

A.9.1 Convergence of Information-Regularized Learning

Proposition A.9 (Convergence to local optimum). Standard stochastic-approximation arguments (Robbins–Monro with $\alpha_t = O(1/\sqrt{t})$, bounded rewards, L-Lipschitz policies, and unbiased, bounded-variance gradient estimators) yield almost-sure convergence to a local optimum of

$$\mathcal{L} = \mathbb{E}[R] - \lambda_{\text{red}} \sum_{i<j} I(\hat{Z}_i; \hat{Z}_j \mid R) - \lambda_{\text{cap}} \sum_i I(\hat{Z}_i; O_i).$$

We do not give a self-contained proof here—the result is a direct application of standard stochastic-approximation theorems, modulo verifying that the neural MI estimators (MINE / InfoNCE) have the bounded-variance property required in expectation. In practice this is what fails first: high-variance MI estimators are a well-known stability problem and are discussed under "Common pitfalls" in Section 6.

A.10 Extensions and Generalizations

A.10.1 Continuous Action Spaces

For continuous action spaces $\mathcal{A}_i = \mathbb{R}^d$, replace discrete entropy with differential entropy:

$$h(\mathcal{Z}_i) = -\int_{\mathbb{R}^d} \mathcal{Z}_i(a|s) \log \mathcal{Z}_i(a|s) da$$

Proposition A.3 (Continuous Capacity Bound): For continuous actions with bounded support $[-M, M]^d$:

$$|\theta_i|_{\min} \approx \frac{h(\mathcal{Z}_i|\mathcal{Z}_{-i}) + d\log(2M)}{\alpha \cdot b}$$

The additional $d\log(2M)$ term accounts for discretization error.

A.10.2 Non-Stationary Environments

Definition A.17 (Time-Varying Optimal Distribution): In non-stationary environments:

$$\mathcal{Z}_t(a|s) = \frac{\exp(\beta Q_t^*(s,a))}{\sum_{a'} \exp(\beta Q_t^*(s,a'))}$$

where Q_t^* changes over time.

Conjecture A.10 (Tracking regret). For slowly changing environments with $\|Q_t^* - Q_{t-1}^*\| \le \epsilon$ in a suitable norm, regret should decompose into a stationary-learning term and an environment-drift term:

$$\text{Regret}_T \lesssim \underbrace{\widetilde{O}\big(\sqrt{T \cdot H(A^*)}\big)}_{\text{learning}} + \underbrace{O(T\epsilon)}_{\text{tracking}}.$$

We state this as a conjecture; a clean derivation would require choosing a specific online learning regret framework (e.g., dynamic regret with shifting comparators) and a coupling between $\|Q_t^* - Q_{t-1}^*\|$ and the entropy term, neither of which we develop here.

A.11 Summary of Conjectures and Heuristics

Claim	Form	Status
Capacity proxy (A.1)	$R_{\pi_i} \lesssim \alpha b m$	Heuristic; report 4 proxies
Lower bound (A.2)	$I(A_i^*; A_i \mid S) \gtrsim H(A_i^* \mid A_{-i}^*) - \delta(\epsilon)$	Conjecture; depends on Lipschitz-in-KL
Sample complexity (A.4)	$m_i = \tilde{O}\big((H + \mathcal{C}(\Pi_i))/\epsilon^2\big)$	Scaling hypothesis
Capacity threshold (A.6)	$\|\theta_i\|_{\min} \approx H/(\alpha b)$	Proxy; sets sweep center
Communication threshold (A.8)	$\sum H(A_i^* \mid O_i) > \sum H(A_i^* \mid A_{-i}^*)$	Conjecture; necessary condition
Convergence (A.9)	Local optimum under SA conditions	Standard SA argument

None of these are theorems in this draft. The v1 "theorems" overstated what we can derive; here we restate the same content as conjectures and heuristics so that the experimental protocol of Section 7 has unambiguous falsifiers.

B Implementation Details

This appendix provides complete implementation details for practitioners wanting to apply the InfoMARL framework. We include full code examples, parameter tuning guidelines, computational optimizations, and troubleshooting guides.

B.1 Complete InfoMARL Implementation

B.1.1 Core Framework Architecture

```python
import torch
import torch.nn as nn
import torch.optim as optim
import numpy as np
from collections import defaultdict, deque
from typing import Dict, List, Tuple, Optional
import logging

class InfoMARLFramework:
    """
    Complete implementation of Information-Theoretic Multi-Agent RL

    This framework provides:
    - Automatic capacity estimation
    - Information-theoretic regularization
    - Adaptive network scaling
    - Convergence monitoring
    - Distributed training support
    """

    def __init__(
        self,
        env,
        n_agents: int,
        obs_dims: List[int],
        action_dims: List[int],
        base_algorithm: str = 'PPO',
        estimate_entropy: bool = True,
        device: str = 'cuda' if torch.cuda.is_available() else 'cpu'
    ):
        """
        Initialize the InfoMARL framework

        Args:
            env: Multi-agent environment (must have step, reset methods)
            n_agents: Number of agents
            obs_dims: List of observation dimensions for each agent
            action_dims: List of action dimensions for each agent
            base_algorithm: Base RL algorithm to use
            estimate_entropy: Whether to estimate conditional entropy for capacity
            device: Computing device (cpu/cuda)
        """
        self.env = env
        self.n_agents = n_agents
        self.obs_dims = obs_dims
        self.action_dims = action_dims
        self.device = device

        # Logging
        self.logger = self._setup_logging()
```

```
        # Estimate information quantities if requested
        if estimate_entropy:
            self.logger.info("Estimating conditional entropies...")
            self.conditional_entropies = self._estimate_conditional_entropies()
        else:
            # Use default values
            self.conditional_entropies = [2.0] * n_agents

        # Create agents with appropriate capacity
        self.agents = self._create_agents(base_algorithm)

        # Initialize information estimators
        self.mi_estimators = self._create_mi_estimators()
        self.relevance_estimators = self._create_relevance_estimators()

        # Hyperparameters (can be modified)
        self.hyperparams = {
            'lambda_redundancy': 0.1,
            'lambda_relevance': 0.1,
            'lambda_capacity': 0.01,
            'learning_rate': 3e-4,
            'batch_size': 256,
            'update_frequency': 10,
            'gamma': 0.99,
            'clip_epsilon': 0.2, # For PPO
            'entropy_coef': 0.01,
            'max_grad_norm': 0.5
        }

        # Tracking
        self.metrics = defaultdict(lambda: deque(maxlen=1000))
        self.episode_count = 0
        self.total_steps = 0

    def _estimate_conditional_entropies(self) -> List[float]:
        """
        Estimate H(Z_i | Z_{-i}) for each agent

        Returns:
            List of conditional entropy estimates
        """
        self.logger.info("Collecting trajectories for entropy estimation...")

        # Collect trajectories with random policies
        trajectories = []
        for _ in range(100): # 100 episodes
            obs = self.env.reset()
            episode = []

            for _ in range(100): # Max 100 steps per episode
                # Random actions
                actions = [np.random.randint(0, self.action_dims[i])
                           for i in range(self.n_agents)]

                next_obs, rewards, dones, info = self.env.step(actions)

                episode.append({
                    'obs': obs,
                    'actions': actions,
                    'rewards': rewards,
                    'next_obs': next_obs,
                    'dones': dones
                })

                obs = next_obs
                if all(dones):
                    break
```

```
            trajectories.append(episode)

        # Estimate entropies
        conditional_entropies = []

        for i in range(self.n_agents):
            # Extract all actions
            all_actions = []
            for traj in trajectories:
                for step in traj:
                    all_actions.append(step['actions'])

            # Compute joint entropy
            joint_actions = [tuple(a) for a in all_actions]
            H_joint = self._estimate_entropy(joint_actions)

            # Compute entropy without agent i
            others_actions = [tuple(a[j] for j in range(len(a)) if j != i)
                              for a in all_actions]
            H_others = self._estimate_entropy(others_actions)

            # Conditional entropy
            H_conditional = H_joint - H_others
            conditional_entropies.append(H_conditional)

            self.logger.info(f"Agent {i}: H(Z_i|Z_{{-i}}) ~ {H_conditional:.2f} bits")

        return conditional_entropies

    def _estimate_entropy(self, samples: List) -> float:
        """
        Estimate entropy from samples using Miller-Madow correction

        Args:
            samples: List of samples

        Returns:
            Entropy estimate in bits
        """
        from collections import Counter

        n = len(samples)
        if n == 0:
            return 0.0

        counts = Counter(samples)
        k = len(counts) # Number of unique values

        # Compute entropy
        H = 0.0
        for count in counts.values():
            p = count / n
            if p > 0:
                H -= p * np.log2(p)

        # Miller-Madow bias correction
        if n > 0:
            H += (k - 1) / (2 * n)

        return H
```

B.1.2 PPO Agent Implementation

```
class PPOAgent(nn.Module):
    """
```

```
    Proximal Policy Optimization agent with adaptive capacity
    """

    def __init__(
        self,
        obs_dim: int,
        action_dim: int,
        network_params: int,
        device: str = 'cpu'
    ):
        super().__init__()

        self.obs_dim = obs_dim
        self.action_dim = action_dim
        self.device = device

        # Calculate hidden dimensions to achieve target parameters
        # For a 2-layer network: params ~ (obs_dim * h1) + (h1 * h2) + (h2 * action_dim)
        # Assuming h1 = h2 = h for simplicity
        h = int(np.sqrt(network_params / 3))

        # Actor network (policy)
        self.actor = nn.Sequential(
            nn.Linear(obs_dim, h),
            nn.ReLU(),
            nn.LayerNorm(h),
            nn.Linear(h, h),
            nn.ReLU(),
            nn.LayerNorm(h),
            nn.Linear(h, action_dim)
        ).to(device)

        # Critic network (value function)
        self.critic = nn.Sequential(
            nn.Linear(obs_dim, h),
            nn.ReLU(),
            nn.LayerNorm(h),
            nn.Linear(h, h),
            nn.ReLU(),
            nn.LayerNorm(h),
            nn.Linear(h, 1)
        ).to(device)

        # Optimizers
        self.actor_optimizer = optim.Adam(self.actor.parameters(), lr=3e-4)
        self.critic_optimizer = optim.Adam(self.critic.parameters(), lr=3e-4)

        # PPO specific
        self.clip_epsilon = 0.2
        self.entropy_coef = 0.01

        # Tracking
        self.training_steps = 0

    def forward(self, obs: torch.Tensor) -> Tuple[torch.Tensor, torch.Tensor]:
        action_logits = self.actor(obs)
        value = self.critic(obs)
        return action_logits, value

    def get_action(self, obs: np.ndarray, deterministic: bool = False) -> int:
        obs_tensor = torch.FloatTensor(obs).unsqueeze(0).to(self.device)

        with torch.no_grad():
            action_logits, _ = self.forward(obs_tensor)
            action_probs = torch.softmax(action_logits, dim=-1)

        if deterministic:
            action = action_probs.argmax().item()
```

```
        else:
            action = torch.multinomial(action_probs, 1).item()

        return action

    def update(
        self,
        obs_batch: torch.Tensor,
        action_batch: torch.Tensor,
        return_batch: torch.Tensor,
        advantage_batch: torch.Tensor,
        old_log_probs: torch.Tensor
    ) -> Dict[str, float]:
        # Get current predictions
        action_logits, values = self.forward(obs_batch)
        action_probs = torch.softmax(action_logits, dim=-1)

        # Compute log probabilities
        action_log_probs = torch.log_softmax(action_logits, dim=-1)
        selected_log_probs = action_log_probs.gather(
            1, action_batch.unsqueeze(1)
        ).squeeze(1)

        # PPO clipped objective
        ratio = torch.exp(selected_log_probs - old_log_probs)
        clipped_ratio = torch.clamp(ratio, 1 - self.clip_epsilon, 1 + self.clip_epsilon)

        actor_loss = -torch.min(
            ratio * advantage_batch,
            clipped_ratio * advantage_batch
        ).mean()

        # Entropy bonus
        entropy = -(action_probs * action_log_probs).sum(dim=-1).mean()
        actor_loss -= self.entropy_coef * entropy

        # Value loss
        value_loss = nn.MSELoss()(values.squeeze(), return_batch)

        # Update networks
        self.actor_optimizer.zero_grad()
        actor_loss.backward()
        nn.utils.clip_grad_norm_(self.actor.parameters(), 0.5)
        self.actor_optimizer.step()

        self.critic_optimizer.zero_grad()
        value_loss.backward()
        nn.utils.clip_grad_norm_(self.critic.parameters(), 0.5)
        self.critic_optimizer.step()

        self.training_steps += 1

        return {
            'actor_loss': actor_loss.item(),
            'value_loss': value_loss.item(),
            'entropy': entropy.item()
        }

    def get_capacity(self) -> int:
        """Get total number of parameters"""
        total = 0
        for p in self.parameters():
            total += p.numel()
        return total

    def get_effective_capacity(self) -> float:
        threshold = 0.01
        effective = 0
```

```
        for p in self.parameters():
            effective += (p.abs() > threshold).sum().item()

        return effective
```

B.2 Information Estimators

B.2.1 Mutual Information Estimator

```
class MutualInformationEstimator(nn.Module):
    """
    Neural estimator for mutual information using MINE
    (Mutual Information Neural Estimation)

    Reference: Belghazi et al., "MINE: Mutual Information Neural Estimation", ICML 2018
    """

    def __init__(
        self,
        dim_x: int,
        dim_y: int,
        hidden_dim: int = 256,
        device: str = 'cpu'
    ):
        super().__init__()

        self.device = device

        # Discriminator network T(x,y)
        self.network = nn.Sequential(
            nn.Linear(dim_x + dim_y, hidden_dim),
            nn.ReLU(),
            nn.BatchNorm1d(hidden_dim),
            nn.Linear(hidden_dim, hidden_dim),
            nn.ReLU(),
            nn.BatchNorm1d(hidden_dim),
            nn.Linear(hidden_dim, 1)
        ).to(device)

        # Optimizer
        self.optimizer = optim.Adam(self.parameters(), lr=1e-3)

        # Moving averages for stability
        self.ma_et = 1.0
        self.ma_rate = 0.01

        # History for variance reduction
        self.history_size = 20
        self.mi_history = deque(maxlen=self.history_size)

    def forward(self, x: torch.Tensor, y: torch.Tensor) -> torch.Tensor:
        xy = torch.cat([x, y], dim=-1)
        return self.network(xy)

    def estimate_mi(
        self,
        x_samples: torch.Tensor,
        y_samples: torch.Tensor,
        n_iterations: int = 5
    ) -> float:
        batch_size = x_samples.shape[0]

        for _ in range(n_iterations):
            # Joint samples from P(X,Y)
            joint_scores = self.forward(x_samples, y_samples)
```

```
            # Marginal samples from P(X)P(Y) - shuffle Y
            y_shuffle = y_samples[torch.randperm(batch_size)]
            marginal_scores = self.forward(x_samples, y_shuffle)

            # MINE-f divergence estimator
            mi_lower_bound = (
                joint_scores.mean() -
                torch.exp(marginal_scores - 1).mean()
            )

            # DV bound for stability
            mi_dv = joint_scores.mean() - torch.log(torch.exp(marginal_scores).mean() + 1e-8)

            # Use combination for robustness
            mi_estimate = 0.5 * mi_lower_bound + 0.5 * mi_dv

            # Update discriminator (maximize MI estimate)
            loss = -mi_estimate

            self.optimizer.zero_grad()
            loss.backward()
            nn.utils.clip_grad_norm_(self.parameters(), 1.0)
            self.optimizer.step()

            with torch.no_grad():
                marginal_exp = torch.exp(marginal_scores).mean().item()
                self.ma_et = (1 - self.ma_rate) * self.ma_et + self.ma_rate * marginal_exp

        with torch.no_grad():
            joint_scores = self.forward(x_samples, y_samples)
            y_shuffle = y_samples[torch.randperm(batch_size)]
            marginal_scores = self.forward(x_samples, y_shuffle)

            mi_final = joint_scores.mean() - torch.log(torch.tensor(self.ma_et))

        self.mi_history.append(mi_final.item())

        if len(self.mi_history) > 0:
            return np.mean(self.mi_history)
        else:
            return mi_final.item()
```

B.2.2 Relevance Estimator

```
class RelevanceEstimator(nn.Module):
    """
    Estimate I(observation; action | reward)
    Measures how much task-relevant information the policy preserves
    """

    def __init__(
        self,
        obs_dim: int,
        action_dim: int,
        hidden_dim: int = 128,
        device: str = 'cpu'
    ):
        super().__init__()

        self.device = device

        self.predictor = nn.Sequential(
            nn.Linear(obs_dim + action_dim, hidden_dim),
            nn.ReLU(),
            nn.BatchNorm1d(hidden_dim),
```

```
            nn.Linear(hidden_dim, hidden_dim),
            nn.ReLU(),
            nn.BatchNorm1d(hidden_dim),
            nn.Linear(hidden_dim, 1)
        ).to(device)

        self.optimizer = optim.Adam(self.parameters(), lr=1e-3)

    def estimate_relevance(
        self,
        obs_batch: torch.Tensor,
        action_batch: torch.Tensor,
        reward_batch: torch.Tensor,
        n_iterations: int = 10
    ) -> float:
        if len(action_batch.shape) == 1:
            action_one_hot = torch.zeros(
                action_batch.shape[0],
                self.predictor[0].in_features - obs_batch.shape[1]
            ).to(self.device)
            action_one_hot.scatter_(1, action_batch.unsqueeze(1), 1)
        else:
            action_one_hot = action_batch

        for _ in range(n_iterations):
            oa_pairs = torch.cat([obs_batch, action_one_hot], dim=-1)
            predicted_rewards = self.predictor(oa_pairs).squeeze()
            loss = nn.MSELoss()(predicted_rewards, reward_batch)

            self.optimizer.zero_grad()
            loss.backward()
            nn.utils.clip_grad_norm_(self.parameters(), 1.0)
            self.optimizer.step()

        with torch.no_grad():
            oa_pairs = torch.cat([obs_batch, action_one_hot], dim=-1)
            predicted_rewards = self.predictor(oa_pairs).squeeze()
            mse = nn.MSELoss()(predicted_rewards, reward_batch)

            reward_var = reward_batch.var()
            relevance = 0.5 * torch.log(reward_var / (mse + 1e-8))

        return max(0, relevance.item())
```

B.3 Training Loop Implementation

```
class InfoMARLTrainer:
    """
    Main training loop for InfoMARL
    """

    def __init__(self, framework: InfoMARLFramework):
        self.framework = framework
        self.replay_buffer = MultiAgentReplayBuffer(capacity=100000)

    def train(
        self,
        n_episodes: int = 10000,
        max_steps_per_episode: int = 100,
        save_frequency: int = 100
    ):
        for episode in range(n_episodes):
            trajectory = self.collect_episode(max_steps_per_episode)
            self.replay_buffer.add_trajectory(trajectory)

            if len(self.replay_buffer) >= self.framework.hyperparams['batch_size']:
```

```
            losses = self.update_agents()
            self.log_metrics(episode, trajectory, losses)

        if episode % save_frequency == 0:
            self.save_checkpoint(episode)

        if episode % 100 == 0:
            self.adapt_difficulty(episode)

def collect_episode(self, max_steps: int) -> List[Dict]:
    obs = self.framework.env.reset()
    trajectory = []

    for step in range(max_steps):
        actions = []
        action_probs = []

        for i, agent in enumerate(self.framework.agents):
            action = agent.get_action(obs[i])
            actions.append(action)

            with torch.no_grad():
                obs_tensor = torch.FloatTensor(obs[i]).unsqueeze(0)
                logits, _ = agent(obs_tensor)
                probs = torch.softmax(logits, dim=-1)
                action_probs.append(probs[0, action].item())

        next_obs, rewards, dones, info = self.framework.env.step(actions)

        trajectory.append({
            'obs': obs,
            'actions': actions,
            'action_probs': action_probs,
            'rewards': rewards,
            'next_obs': next_obs,
            'dones': dones,
            'info': info
        })

        obs = next_obs

        if all(dones):
            break

    return trajectory

def update_agents(self) -> Dict[str, float]:
    batch = self.replay_buffer.sample(
        self.framework.hyperparams['batch_size']
    )

    all_losses = {}

    for i, agent in enumerate(self.framework.agents):
        obs_batch = torch.FloatTensor(batch['obs'][:, i])
        action_batch = torch.LongTensor(batch['actions'][:, i])
        reward_batch = torch.FloatTensor(batch['rewards'][:, i])

        returns, advantages = self.compute_gae(
            reward_batch,
            batch['values'][:, i] if 'values' in batch else None
        )

        rl_losses = agent.update(
            obs_batch,
            action_batch,
            returns,
            advantages,
```

```
                batch['old_log_probs'][:, i]
            )

            info_losses = self.compute_info_regularization(i, batch)

            for key, value in {**rl_losses, **info_losses}.items():
                all_losses[f'agent_{i}_{key}'] = value

        return all_losses

    def compute_info_regularization(
        self,
        agent_idx: int,
        batch: Dict
    ) -> Dict[str, float]:
        losses = {}

        # Redundancy penalty
        redundancy = 0
        for j in range(self.framework.n_agents):
            if j != agent_idx:
                key = (min(agent_idx, j), max(agent_idx, j))
                if key in self.framework.mi_estimators:
                    actions_i = torch.FloatTensor(batch['actions'][:, agent_idx])
                    actions_j = torch.FloatTensor(batch['actions'][:, j])

                    mi = self.framework.mi_estimators[key].estimate_mi(
                        actions_i, actions_j
                    )
                    redundancy += mi

        losses['redundancy'] = redundancy * self.framework.hyperparams['lambda_redundancy']

        # Relevance bonus
        relevance_estimator = self.framework.relevance_estimators[agent_idx]
        obs = torch.FloatTensor(batch['obs'][:, agent_idx])
        actions = torch.LongTensor(batch['actions'][:, agent_idx])
        rewards = torch.FloatTensor(batch['rewards'][:, agent_idx])

        relevance = relevance_estimator.estimate_relevance(
            obs, actions, rewards
        )
        losses['relevance'] = -relevance * self.framework.hyperparams['lambda_relevance']

        # Capacity penalty
        agent = self.framework.agents[agent_idx]
        capacity = agent.get_effective_capacity()
        target_capacity = self.framework.conditional_entropies[agent_idx] * 100

        capacity_penalty = max(0, capacity - target_capacity) / target_capacity
        losses['capacity'] = capacity_penalty * self.framework.hyperparams['lambda_capacity']

        return losses
```

B.4 Hyperparameter Tuning

```
class HyperparameterTuner:
    """
    Automated hyperparameter tuning for InfoMARL
    """

    def __init__(self, env_factory, n_trials: int = 50):
        self.env_factory = env_factory
        self.n_trials = n_trials

    def objective(self, trial):
```

```
        hyperparams = {
            'lambda_redundancy': trial.suggest_loguniform('lambda_redundancy', 1e-3, 1.0),
            'lambda_relevance': trial.suggest_loguniform('lambda_relevance', 1e-3, 1.0),
            'lambda_capacity': trial.suggest_loguniform('lambda_capacity', 1e-4, 1e-1),
            'learning_rate': trial.suggest_loguniform('learning_rate', 1e-5, 1e-2),
            'batch_size': trial.suggest_categorical('batch_size', [64, 128, 256, 512]),
            'entropy_coef': trial.suggest_loguniform('entropy_coef', 1e-4, 1e-1),
            'clip_epsilon': trial.suggest_uniform('clip_epsilon', 0.1, 0.3)
        }

        env = self.env_factory()
        framework = InfoMARLFramework(env, n_agents=4)
        framework.hyperparams.update(hyperparams)

        trainer = InfoMARLTrainer(framework)
        trainer.train(n_episodes=1000)

        performance = self.evaluate(framework)

        return performance

    def tune(self):
        import optuna

        study = optuna.create_study(
            direction='maximize',
            sampler=optuna.samplers.TPESampler(),
            pruner=optuna.pruners.MedianPruner()
        )

        study.optimize(self.objective, n_trials=self.n_trials)

        print(f"Best trial: {study.best_trial.number}")
        print(f"Best value: {study.best_value}")
        print(f"Best params: {study.best_params}")

        return study.best_params

    def evaluate(self, framework: InfoMARLFramework, n_episodes: int = 100):
        total_reward = 0

        for _ in range(n_episodes):
            obs = framework.env.reset()
            episode_reward = 0

            while True:
                actions = [agent.get_action(obs[i], deterministic=True)
                           for i, agent in enumerate(framework.agents)]

                obs, rewards, dones, _ = framework.env.step(actions)
                episode_reward += sum(rewards)

                if all(dones):
                    break

            total_reward += episode_reward

        return total_reward / n_episodes
```

B.5 Computational Optimizations

```
class OptimizedInfoMARL:
    """
    Optimized version with various computational tricks
    """
```

```
    @staticmethod
    def batch_mi_estimation(action_matrix: np.ndarray) -> np.ndarray:
        n_agents = action_matrix.shape[1]
        mi_matrix = np.zeros((n_agents, n_agents))

        for i in range(n_agents):
            for j in range(i+1, n_agents):
                joint = np.concatenate([
                    action_matrix[:, i],
                    action_matrix[:, j]
                ], axis=1)

                H_joint = OptimizedInfoMARL._fast_entropy(joint)
                H_i = OptimizedInfoMARL._fast_entropy(action_matrix[:, i])
                H_j = OptimizedInfoMARL._fast_entropy(action_matrix[:, j])

                mi_matrix[i, j] = H_i + H_j - H_joint
                mi_matrix[j, i] = mi_matrix[i, j]

        return mi_matrix

    @staticmethod
    def _fast_entropy(data: np.ndarray) -> float:
        unique, counts = np.unique(data, return_counts=True, axis=0)
        probs = counts / len(data)
        return -np.sum(probs * np.log2(probs + 1e-10))

    @staticmethod
    @torch.jit.script
    def jit_compute_advantages(
        rewards: torch.Tensor,
        values: torch.Tensor,
        gamma: float = 0.99,
        gae_lambda: float = 0.95
    ) -> Tuple[torch.Tensor, torch.Tensor]:
        advantages = torch.zeros_like(rewards)
        returns = torch.zeros_like(rewards)

        gae = 0
        next_value = 0

        for t in reversed(range(len(rewards))):
            delta = rewards[t] + gamma * next_value - values[t]
            gae = delta + gamma * gae_lambda * gae
            advantages[t] = gae
            returns[t] = advantages[t] + values[t]
            next_value = values[t]

        return returns, advantages
```

B.6 Troubleshooting Guide

```
class InfoMARLDiagnostics:
    """
    Diagnostic tools for debugging InfoMARL training
    """

    @staticmethod
    def diagnose_convergence_issues(framework: InfoMARLFramework):
        issues = []
        recommendations = []

        # Check capacity
        for i, agent in enumerate(framework.agents):
            capacity = agent.get_capacity()
            required = framework.conditional_entropies[i] * 100
```

```
        if capacity < required * 0.8:
            issues.append(f"Agent {i} undercapacity: {capacity} < {required}")
            recommendations.append(f"Increase agent {i} network size")
        elif capacity > required * 3:
            issues.append(f"Agent {i} overcapacity: {capacity} >> {required}")
            recommendations.append(f"Add stronger capacity regularization")

    # Check learning rates
    for i, agent in enumerate(framework.agents):
        lr = agent.actor_optimizer.param_groups[0]['lr']
        if lr > 1e-2:
            issues.append(f"Agent {i} learning rate too high: {lr}")
            recommendations.append("Reduce learning rate")
        elif lr < 1e-6:
            issues.append(f"Agent {i} learning rate too low: {lr}")
            recommendations.append("Increase learning rate")

    # Check MI estimates
    total_mi = 0
    for key, estimator in framework.mi_estimators.items():
        if len(estimator.mi_history) > 0:
            avg_mi = np.mean(estimator.mi_history)
            if avg_mi > 0.9:
                issues.append(f"High redundancy between agents {key}: {avg_mi}")
                recommendations.append("Increase lambda_redundancy")
            total_mi += avg_mi

    # Check gradient norms
    for i, agent in enumerate(framework.agents):
        total_norm = 0
        for p in agent.parameters():
            if p.grad is not None:
                param_norm = p.grad.data.norm(2)
                total_norm += param_norm.item() ** 2
        total_norm = total_norm ** 0.5

        if total_norm > 100:
            issues.append(f"Agent {i} gradient explosion: {total_norm}")
            recommendations.append("Reduce learning rate or increase gradient clipping")
        elif total_norm < 1e-6:
            issues.append(f"Agent {i} vanishing gradients: {total_norm}")
            recommendations.append("Check network initialization")

    print("\n=== InfoMARL Diagnostic Report ===")
    print(f"Found {len(issues)} potential issues:\n")

    for issue, rec in zip(issues, recommendations):
        print(f"Issue: {issue}")
        print(f"Recommendation: {rec}\n")

    if len(issues) == 0:
        print("No obvious issues found. Training may just need more time.")

    return issues, recommendations
```

B.7 Summary

This appendix has provided complete implementation details for the InfoMARL framework, including:

1. **Full framework architecture** with automatic capacity estimation
2. **Complete agent implementations** (PPO example)
3. **Information estimators** (MINE and relevance)

4. **Training loop** with all components integrated
5. **Hyperparameter tuning** using Optuna
6. **Computational optimizations** for efficiency
7. **Diagnostic tools** for troubleshooting

The code is designed to be modular, extensible, and production-ready. Practitioners can use these implementations as a starting point and customize them for their specific multi-agent environments.

C The Bucket Brigade Environment — Complete Specification

C.1 Environment Overview and Motivation

The Bucket Brigade environment was specifically designed to test the predictions of our information-theoretic framework. It provides a multi-agent firefighting scenario where coordination is essential but communication is limited, making it ideal for studying distributed encoding of optimal strategies.

C.1.1 Why Bucket Brigade?

We chose to develop Bucket Brigade because it offers several crucial properties:

1. **Tunable Correlation**: Fire spread creates natural correlation between agents' observations
2. **Measurable Optimality**: We can compute oracle policies for comparison
3. **Mixed Incentives**: Individual and team rewards create interesting trade-offs
4. **Scalable Complexity**: From trivial to intractable through parameter adjustment
5. **Discrete and Continuous**: Supports both action types for broad applicability

C.2 Complete Game Mechanics

C.2.1 World State Representation

```python
class BucketBrigadeWorld:
    """
    Complete world state for Bucket Brigade
    """

    def __init__(self, n_houses: int = 10, n_agents: int = 4):
        # World configuration
        self.n_houses = n_houses
        self.n_agents = n_agents

        # House states
        self.SAFE = 0
        self.BURNING = 1
        self.RUINED = 2

        # Initialize ring topology
        self.houses = np.zeros(n_houses, dtype=int)

        # Agent ownership (each agent owns houses)
        self.ownership = np.array([i % n_agents for i in range(n_houses)])

        # Agent properties
        self.agent_positions = np.zeros(n_agents, dtype=int)
        self.agent_energy = np.ones(n_agents) * 100.0
        self.agent_mode = ['idle'] * n_agents # idle, working, resting
```

```
        # Dynamics parameters
        self.p_spread = 0.3 # Fire spread probability
        self.p_ignite = 0.05 # Spontaneous ignition probability
        self.p_solo = 0.6 # Solo extinguish success probability
        self.burnout_time = 5 # Steps until house ruins

        # Reward parameters
        self.team_reward_per_safe = 1.0
        self.own_house_bonus = 5.0
        self.work_cost_penalty = 0.5
        self.rest_recovery = 10.0

        # History tracking
        self.time_step = 0
        self.total_houses_saved = 0
        self.total_houses_lost = 0
        self.burn_duration = np.zeros(n_houses)
```

C.2.2 Turn Sequence Details

Each turn consists of nine phases executed in strict order:

Phase 1: Observation Generation

```
def generate_observations(self, obs_radius: int = 2) -> List[np.ndarray]:
    """
    Generate partial observations for each agent

    Args:
        obs_radius: How many houses each agent can see

    Returns:
        List of observation arrays
    """
    observations = []

    for agent_id in range(self.n_agents):
        # Get agent's position
        pos = self.agent_positions[agent_id]

        # Visible houses (with wraparound)
        visible_indices = []
        for offset in range(-obs_radius, obs_radius + 1):
            idx = (pos + offset) % self.n_houses
            visible_indices.append(idx)

        # Construct observation
        obs = {
            'house_states': self.houses[visible_indices],
            'house_ownership': self.ownership[visible_indices],
            'own_energy': self.agent_energy[agent_id],
            'own_position': pos,
            'time_step': self.time_step,
            'houses_on_fire': np.sum(self.houses == self.BURNING),
            'houses_ruined': np.sum(self.houses == self.RUINED)
        }

        # Flatten to vector
        obs_vector = self._flatten_observation(obs)
        observations.append(obs_vector)

    return observations
```

Phase 2-9: Complete Turn Execution

1. **Signal Broadcasting**: Optional cheap-talk phase

2. **Action Execution**: Agents move and choose work/rest
3. **Fire Extinguishing**: Success probability $P_{\text{ext}} = 1 - (1 - p_{\text{solo}})^{n_{\text{workers}}}$
4. **Fire Burn-out**: Houses burning too long become ruined
5. **Fire Spread**: Fires spread to neighbors with probability p_{spread}
6. **Random Ignition**: Safe houses may catch fire with probability p_{ignite}
7. **Reward Calculation**: Team and individual rewards computed
8. **Check Termination**: Episode ends when all houses resolved or timeout

C.3 Parameterizable Scenarios

Twelve canonical scenarios span cooperation regimes:

Category	Examples	Purpose
Baselines	`default`, `easy`, `hard`	Tune overall difficulty and stochasticity
Pure Cooperation	`trivial_cooperation`	Zero incentive misalignment
Dilemma Variants	`greedy_neighbor`, `sparse_heroics`, `rest_trap`	Manipulate cost-benefit structure
Coordination Stress	`chain_reaction`, `overcrowding`	Require distributed sub-tasking
Signaling Tests	`deceptive_calm`, `mixed_motivation`	Examine cheap-talk value

These scenarios expose different conditional-entropy regimes — from nearly deterministic cooperation to highly uncertain coordination.

C.4 Mapping to Information-Theoretic Quantities

Game Element	Theoretical Analogue
Local observation o_i	Correlated source sample
Agent policy $\pi_i(a_i \mid o_i)$	Encoder
Environment transition + reward	Joint decoder
Centralized oracle policy $\pi^*(a \mid s)$	Latent optimal action distribution $\mathcal{Z}$
Cheap-talk channel	Optional Wyner–Ziv side information
Fire spread correlation	Degree of inter-agent dependence $I(o_i; o_j)$

This mapping allows direct computation of:

- $H(\mathcal{Z}_i)$, $H(\mathcal{Z}_i \mid \mathcal{Z}_{-i})$ — entropy of optimal actions
- $R_{\pi_i} \approx I(\mathcal{Z}_i; \pi_i)$ — policy information rate
- $I(\pi_i; \pi_j)$ — redundancy between agents' encodings

C.5 Experimental Objectives

1. **Validate the Distributed Learning Conjecture**
    - Train decentralized agents of varying network capacity
    - Measure R_{π_i} and $H(\mathcal{Z}_i \mid \mathcal{Z}_{-i})$
    - Test whether coordination quality rises once $R_{\pi_i} \gtrsim H(\mathcal{Z}_i \mid \mathcal{Z}_{-i})$
2. **Explore Observation Correlation Effects**

- Mask local observations to reduce $I(o_i; o_j)$
- Evaluate degradation of team reward as correlation decreases

3. **Detect Emergent Specialization**
   - Monitor $I(\pi_i; \pi_j)$ over training
   - Expect decline as agents divide encoding responsibilities

C.6 Metrics and Analysis

For each episode, log tuples $(t, s_t, o_i^t, a_i^t, r_i^t, a_i^{*,t})$ where $a_i^{*,t}$ is the oracle optimal action.

From these samples, compute:

Metric	Definition
$H(\mathcal{Z}_i)$	Empirical entropy of oracle actions
$H(\mathcal{Z}_i \mid \mathcal{Z}_{-i})$	Conditional entropy given others' oracle actions
R_{π_i}	MI between policy actions and oracle actions
$I(\pi_i; \pi_j)$	MI between agents' actions over time
Reward variance	Indicator of coordination stability

Analysis modules (Rust + Python FFI) will export logs as CSV for offline computation using discrete entropy estimators.

C.7 Planned Experiments and Predictions

Experiment	Manipulation	Expected Outcome
E1: Capacity Sweep	Vary network size	Performance plateau at $\approx H(\mathcal{Z}_i \mid \mathcal{Z}_{-i})$
E2: Correlation Ablation	Mask observations	Degraded reward $\propto$ reduced $I(o_i; o_j)$
E3: Specialization Emergence	Train under social dilemma	$I(\pi_i; \pi_j)$ decreases over time
E4: Communication Threshold	Enable honest signals	Improves when $H(\mathcal{Z}_i \mid o_i) \gg H(\mathcal{Z}_i \mid \mathcal{Z}_{-i})$

Plots of performance vs information ratios should exhibit Slepian–Wolf–style phase transitions.

C.8 Implementation Notes

- The Rust core ensures deterministic, reproducible dynamics
- A Python interface (`bucket-brigade-analyzer/`) handles entropy estimation and visualization
- Configurable scenario files allow batch runs across difficulty levels
- All code and data will be released under an open license for replication

C.9 Significance

Bucket Brigade provides an interpretable microcosm of distributed decision making. By measuring how coordination emerges as a function of conditional information and network capacity, these experiments will supply the first empirical evidence for the Distributed Policy Learning Conjecture and clarify the information-theoretic limits of cooperation without communication.

D Entropy Estimation and Analysis Methodology

D.1 Information-Theoretic Metrics for Multi-Agent Systems

D.1.1 Core Metrics

For multi-agent systems, we track several key information-theoretic quantities:

1. **Marginal Entropy:** $H(\mathcal{Z}_i)$
   - Measures the complexity/diversity of agent i's optimal behavior
   - High values indicate the agent needs many different strategies
   - Low values suggest simple, repetitive behavior

2. **Conditional Entropy:** $H(\mathcal{Z}_i|\mathcal{Z}_{-i})$
   - Measures unique information agent i must encode
   - This is our key quantity for determining capacity requirements
   - Represents irreducible complexity given perfect coordination

3. **Mutual Information:** $I(\pi_i; \pi_j)$
   - Measures redundancy between learned policies
   - High values indicate agents learning similar strategies
   - Should be minimized for efficient encoding

4. **Policy Information Rate:** $R_{\pi_i} = I(\mathcal{Z}_i; \pi_i)$
   - Measures how much optimal behavior the policy captures
   - Should approach $H(\mathcal{Z}_i|\mathcal{Z}_{-i})$ for good coordination
   - Gap indicates learning inefficiency

D.2 Estimation Procedures

D.2.1 Discrete Entropy Estimation

For discrete distributions (like Bucket Brigade actions), we use plug-in estimators with bias correction:

```
import numpy as np
from collections import Counter
from typing import List, Optional, Union

def estimate_entropy_discrete(
    samples: List,
    bias_correction: str = 'miller-madow'
) -> float:
    """
    Estimate entropy from discrete samples with bias correction

    Args:
        samples: List of samples from distribution
        bias_correction: Type of bias correction to apply
            - 'miller-madow': Standard MM correction
            - 'panzeri-treves': Better for small samples
            - 'bootstrap': Bootstrap bias correction
```

```
            - 'none': No correction

    Returns:
        Entropy estimate in bits
    """
    n = len(samples)
    if n == 0:
        return 0.0

    counts = Counter(samples)
    k = len(counts) # Number of observed categories

    # Plug-in estimate
    H_plugin = 0.0
    for count in counts.values():
        p = count / n
        if p > 0:
            H_plugin -= p * np.log2(p)

    # Apply bias correction
    if bias_correction == 'miller-madow':
        H_corrected = H_plugin + (k - 1) / (2 * n)

    elif bias_correction == 'panzeri-treves':
        H_corrected = H_plugin + (k - 1) / (2 * n) - (k - 1)**2 / (12 * n**2)

    elif bias_correction == 'bootstrap':
        H_corrected = bootstrap_bias_correction(samples, H_plugin, n_bootstrap=100)

    elif bias_correction == 'none':
        H_corrected = H_plugin

    else:
        raise ValueError(f"Unknown bias correction: {bias_correction}")

    return max(0, H_corrected)
```

D.2.2 Continuous Entropy Estimation

For continuous distributions, we use kernel density estimation or k-nearest neighbor methods:

```
def estimate_entropy_continuous(
    samples: np.ndarray,
    method: str = 'knn'
) -> float:
    """
    Estimate differential entropy from continuous samples

    Args:
        samples: Array of samples, shape (n_samples, dim)
        method: Estimation method
            - 'knn': k-Nearest Neighbor (Kozachenko-Leonenko)
            - 'kde': Kernel Density Estimation
            - 'gaussian': Assume Gaussian distribution

    Returns:
        Differential entropy estimate in nats
    """
    n, d = samples.shape

    if method == 'knn':
        from sklearn.neighbors import NearestNeighbors
        from scipy.special import digamma

        k = min(4, n // 10)
        nn = NearestNeighbors(n_neighbors=k+1)
        nn.fit(samples)
```

```
        distances, _ = nn.kneighbors(samples)
        distances = distances[:, -1]

        h = digamma(n) - digamma(k) + d * np.log(2)
        h += d * np.mean(np.log(distances + 1e-10))

        return h

    elif method == 'kde':
        from sklearn.neighbors import KernelDensity

        std = np.std(samples, axis=0)
        bandwidth = 1.06 * np.mean(std) * n**(-1/(d+4))

        kde = KernelDensity(kernel='gaussian', bandwidth=bandwidth)
        kde.fit(samples)

        log_density = kde.score_samples(samples)
        h = -np.mean(log_density)

        return h
```

D.2.3 Conditional Entropy Estimation

Estimating conditional entropy $H(X|Y)$ is more challenging than marginal entropy:

```
def estimate_conditional_entropy(
    x_samples: np.ndarray,
    y_samples: np.ndarray,
    method: str = 'direct'
) -> float:
    """
    Estimate H(X|Y) from samples
    """
    if method == 'direct':
        xy = np.concatenate([x_samples, y_samples], axis=1)
        H_xy = estimate_entropy_continuous(xy)
        H_y = estimate_entropy_continuous(y_samples)
        return H_xy - H_y

    elif method == 'binning':
        from sklearn.preprocessing import KBinsDiscretizer

        n_bins = min(20, len(y_samples) // 50)
        discretizer = KBinsDiscretizer(n_bins=n_bins, encode='ordinal')
        y_discrete = discretizer.fit_transform(y_samples).astype(int).ravel()

        H_conditional = 0
        for y_value in range(n_bins):
            mask = (y_discrete == y_value)
            if np.sum(mask) > 1:
                x_given_y = x_samples[mask]
                p_y = np.sum(mask) / len(y_samples)
                H_x_given_y = estimate_entropy_continuous(x_given_y)
                H_conditional += p_y * H_x_given_y

        return H_conditional
```

D.3 Analysis Pipeline

D.3.1 Complete Analysis Workflow

```
from typing import Dict, List
from collections import defaultdict
```

```python
class InformationTheoreticAnalysis:
    """
    Complete pipeline for analyzing multi-agent environments
    """

    def __init__(self, env, n_samples: int = 10000):
        self.env = env
        self.n_samples = n_samples
        self.results = {}

    def run_complete_analysis(self) -> Dict:
        print("Starting information-theoretic analysis...")

        print("Step 1: Collecting trajectories...")
        trajectories = self.collect_trajectories()

        print("Step 2: Computing marginal entropies...")
        self.compute_marginal_entropies(trajectories)

        print("Step 3: Computing joint entropy...")
        self.compute_joint_entropy(trajectories)

        print("Step 4: Computing conditional entropies...")
        self.compute_conditional_entropies(trajectories)

        print("Step 5: Computing mutual information...")
        self.compute_mutual_information(trajectories)

        print("Step 6: Computing observation-conditioned entropies...")
        self.compute_observation_entropies(trajectories)

        print("Step 7: Computing derived metrics...")
        self.compute_derived_metrics()

        print("Step 8: Generating report...")
        self.generate_report()

        return self.results

    def compute_conditional_entropies(self, trajectories: List[Dict]):
        H_joint = self.results['H_joint']

        for i in range(self.env.n_agents):
            others_actions = [
                tuple(t['actions'][j] for j in range(self.env.n_agents) if j != i)
                for t in trajectories
            ]
            H_others = estimate_entropy_discrete(others_actions)
            H_conditional = H_joint - H_others

            self.results[f'H_{i}_given_others'] = H_conditional
            print(f" Agent {i}: H(Z_{i}|Z_{{-i}}) = {H_conditional:.3f} bits")
```

D.4 Validation and Testing

D.4.1 Synthetic Test Cases

```python
def test_entropy_estimators():
    """
    Test entropy estimators on known distributions
    """
    print("Testing entropy estimators...")

    # Test 1: Uniform distribution
    uniform_samples = np.random.randint(0, 8, size=10000)
```

```
    H_uniform_true = np.log2(8) # 3 bits
    H_uniform_est = estimate_entropy_discrete(uniform_samples)
    print(f"Uniform(8): True={H_uniform_true:.3f}, Est={H_uniform_est:.3f}")
    assert abs(H_uniform_est - H_uniform_true) < 0.1, "Uniform test failed"

    # Test 2: Binomial distribution
    from scipy.stats import binom
    binomial_samples = np.random.binomial(10, 0.3, size=10000)
    pmf = [binom.pmf(k, 10, 0.3) for k in range(11)]
    H_binomial_true = -sum(p * np.log2(p) for p in pmf if p > 0)
    H_binomial_est = estimate_entropy_discrete(binomial_samples)
    print(f"Binomial(10,0.3): True={H_binomial_true:.3f}, Est={H_binomial_est:.3f}")

    # Test 3: Conditional entropy
    x = np.random.randint(0, 4, size=10000)
    noise = np.random.randint(0, 2, size=10000)
    y = (x + noise) % 4

    H_y_given_x_true = 1.0

    H_y_given_x_est = 0
    for x_val in range(4):
        mask = (x == x_val)
        y_given_x = y[mask]
        p_x = np.sum(mask) / len(x)
        H_y_given_x_val = estimate_entropy_discrete(y_given_x.tolist())
        H_y_given_x_est += p_x * H_y_given_x_val

    print(f"H(Y|X) with Y=X+noise: True={H_y_given_x_true:.3f}, Est={H_y_given_x_est:.3f}")

    print("All tests passed!")
    return True
```

D.4.2 Convergence Analysis

```
def analyze_convergence(
    env,
    sample_sizes: List[int] = [100, 500, 1000, 5000, 10000, 50000]
) -> Dict:
    """
    Analyze how entropy estimates converge with sample size
    """
    import time
    results = {size: {} for size in sample_sizes}

    for n_samples in sample_sizes:
        print(f"\nAnalyzing with {n_samples} samples...")
        start_time = time.time()

        analyzer = InformationTheoreticAnalysis(env, n_samples)
        metrics = analyzer.run_complete_analysis()

        results[n_samples] = {
            'H_joint': metrics['H_joint'],
            'H_conditional_avg': metrics['H_conditional_avg'],
            'runtime': time.time() - start_time
        }

        print(f" Runtime: {results[n_samples]['runtime']:.2f} seconds")

    from scipy.optimize import curve_fit

    def convergence_model(n, H_inf, a, b):
        return H_inf - a * np.exp(-b * n)

    sizes = list(results.keys())
```

```
    H_joints = [results[s]['H_joint'] for s in sizes]

    popt_joint, _ = curve_fit(
        convergence_model,
        sizes,
        H_joints,
        p0=[H_joints[-1], 1, 0.0001]
    )

    print(f"\nAsymptotic joint entropy: {popt_joint[0]:.3f} bits")
    print(f"Convergence rate: {popt_joint[2]:.6f}")

    return results
```

D.5 Practical Guidelines

D.5.1 Choosing the Right Estimator

Data Type	Sample Size	Recommended Method	Bias Correction
Discrete	< 100	Plug-in	Panzeri-Treves
Discrete	100-1000	Plug-in	Miller-Madow
Discrete	> 1000	Plug-in	Miller-Madow
Continuous (1D)	Any	KDE	N/A
Continuous (2-5D)	< 1000	Gaussian	N/A
Continuous (2-5D)	> 1000	k-NN	N/A

D.5.2 Sample Size Requirements

For Discrete Distributions:

- Base requirement: $\sim$10 samples per category
- For 0.1 bit error: $n \approx 10k$ where k is number of categories
- For 0.01 bit error: $n \approx 100k$

For Continuous Distributions:

- Exponential in dimension: $n \propto 2^d$
- 1D: $\sim$1000 samples sufficient
- 5D: $\sim$10,000 samples minimum
- 10D+: Consider parametric assumptions

D.6 Summary

This appendix has provided comprehensive methodology for entropy estimation in multi-agent systems:

1. **Core Metrics**: Defined the key information-theoretic quantities
2. **Estimators**: Implemented various estimation methods for discrete and continuous cases
3. **Pipeline**: Complete analysis workflow from data collection to report generation
4. **Validation**: Test cases and convergence analysis
5. **Guidelines**: Practical advice for choosing estimators and sample sizes

These tools enable practitioners to:

- Analyze the information structure of their multi-agent problems
- Determine network capacity requirements
- Predict when communication will be beneficial
- Validate the predictions of the information-theoretic framework

The methods are designed to be robust, efficient, and applicable to a wide range of multi-agent scenarios.

E Extended Proofs and Supplementary Material

E.1 Extended Proofs

E.1.1 Proof of the Distributed Learning Conjecture for Linear Networks

While the general conjecture remains open, we can prove it for the special case of linear networks with Gaussian distributions.

Proposition E.1 (Linear–Gaussian rank bound). For linear policies $\pi_i(o_i) = W_i o_i + b_i$ and a jointly Gaussian optimal action distribution $A^* \sim \mathcal{N}(\mu, \Sigma)$, achieving ϵ-optimal coordination requires

$$\operatorname{rank}(W_i) \;\cdot\; \log \sigma_{\max}(W_i) \;\geq\; H(A_i^* \mid A_{-i}^*) - \delta(\epsilon),$$

where $H(A_i^* \mid A_{-i}^*) = \frac{1}{2}\log\big((2\pi e)^{d_i} \det \Sigma_{i|-i}\big)$ (differential entropy of the conditional Gaussian). The previous draft wrote a closed-form lower bound on $\operatorname{rank}(W_i)$ alone; that form was dimensionally suspect (negative when $\sigma_{\min} > 1$) and depended implicitly on $\sigma_{\max}(W_i)$. The corrected inequality couples rank and spectral norm.

Proof: **Step 1:** Express conditional entropy for Gaussians.
For jointly Gaussian random variables, the conditional entropy is:

$$H(\mathcal{Z}_i|\mathcal{Z}_{-i}) = \frac{1}{2}\log(2\pi e \cdot \det(\Sigma_{i|-i}))$$

where $\Sigma_{i|-i}$ is the conditional covariance matrix.
Step 2: Relate network capacity to rank.
For a linear transformation W_i, the maximum preserved information is:

$$I(o_i; W_i o_i) \leq \operatorname{rank}(W_i) \cdot \log \sigma_{\max}(W_i)$$

Step 3: Apply data processing inequality.
Since $\mathcal{Z}_i \to o_i \to \pi_i(o_i)$ forms a Markov chain:

$$I(\mathcal{Z}_i; \pi_i) \leq I(\mathcal{Z}_i; o_i) \leq I(o_i; W_i o_i)$$

Step 4: Combine bounds.
For ϵ-optimal performance:

$$I(\mathcal{Z}_i; \pi_i) \geq H(\mathcal{Z}_i|\mathcal{Z}_{-i}) - \delta(\epsilon)$$

Therefore:

$$\operatorname{rank}(W_i) \cdot \log \sigma_{\max}(W_i) \geq H(\mathcal{Z}_i|\mathcal{Z}_{-i}) - \delta(\epsilon)$$

Rearranging gives the result. □

E.1.2 Phase Transition Heuristic (was Theorem E.2)

The v1 draft argued, via a mean-field / saddle-point analogy from statistical mechanics, that team reward $J(\theta)$ should exhibit a sigmoid transition in capacity C near the proxy-implied threshold $C^* \approx H(A_i^* \mid A_{-i}^*)/b$, with width $\sigma \propto \sqrt{T}$. We retain this only as a *heuristic motivation* for Prediction 5.1: the experimental test is to look for the knee, not to verify the mean-field calculation. The full statistical-mechanics argument requires technical assumptions (partition function over policy parameters, order parameter identified with a specific MI, saddle-point validity) that we do not justify here. The downgrade is honest about the gap between the heuristic and a real phase-transition theorem.

E.1.3 Specialization Under Regularization (proposition)

Proposition E.3. Consider the conditional-MI regularized objective

$$\pi^* = \arg\min_{\pi} \left[-J(\pi) + \lambda_r \sum_{i<j} I(\pi_i; \pi_j \mid S) \right].$$

As $\lambda_r \to \infty$ with reward unchanged, the optimal policies satisfy $I(\pi_i^*; \pi_j^* \mid S) \to 0$: agents become conditionally independent given the state. This follows from $I \geq 0$ and continuity of the objective in λ_r; it is a structural statement, not a claim about reward at finite λ_r.

Why this is weaker than the v1 claim. The v1 "Theorem E.3" used *unconditional* $I(\pi_i; \pi_j)$, which as PMIC [35] documents can fight tasks requiring synchronization. Conditioning on S excludes that part of MI which the state itself induces, so the penalty pushes only the redundancy that is genuinely wasteful. The price is that at large λ_r the regularized problem may decouple agents in ways that hurt J; the balance is what the empirical sweep in Section 7's P3 protocol is designed to characterize.

E.2 Additional Experimental Details

E.2.1 Hyperparameter Sensitivity Analysis

We conducted extensive hyperparameter sensitivity analysis to understand the robustness of InfoMARL:

```
def hyperparameter_sensitivity_analysis():
    """
    Test sensitivity to key hyperparameters
    """
    import itertools

    param_grid = {
        'lambda_redundancy': [0.01, 0.05, 0.1, 0.2, 0.5],
        'lambda_relevance': [0.01, 0.05, 0.1, 0.2, 0.5],
        'lambda_capacity': [0.001, 0.005, 0.01, 0.02, 0.05],
        'learning_rate': [1e-4, 3e-4, 1e-3, 3e-3],
        'batch_size': [64, 128, 256, 512]
    }

    results = {}
    baseline_performance = ppo_baseline_reward() # measured per-scenario at run time

    for params in itertools.product(*param_grid.values()):
        param_dict = dict(zip(param_grid.keys(), params))

        performance = train_with_params(param_dict)

        if performance > baseline_performance:
            results[str(param_dict)] = performance

    best_params = max(results, key=results.get)
    print(f"Best parameters: {best_params}")
    print(f"Best performance: {results[best_params]}")
```

```
    importance = {}
    for param_name in param_grid.keys():
        param_performances = []
        for key, perf in results.items():
            if param_name in key:
                param_performances.append(perf)

        importance[param_name] = np.var(param_performances)

    ranked_params = sorted(importance.items(), key=lambda x: x[1], reverse=True)
    print("\nParameter importance (by variance):")
    for param, imp in ranked_params:
        print(f" {param}: {imp:.4f}")

    return results, importance
```

Planned sensitivity sweep:

Hyperparameter	Sweep range	Rationale
$\lambda_{\text{redundancy}}$	$\{0, 10^{-3}, 10^{-2}, 10^{-1}\}$	Tests P3 (specialization) ablation
$\lambda_{\text{relevance}}$	$\{0, 10^{-3}, 10^{-2}, 10^{-1}\}$	Tests reward-coupling strength
$\lambda_{\text{capacity}}$	$\{0, 10^{-3}, 10^{-2}\}$	Tests representation IB strength
Learning rate	$\{1\times10^{-4}, 3\times10^{-4}, 1\times10^{-3}\}$	PPO default $\pm$ 0.5 dex
Batch size	$\{64, 128, 256, 512\}$	Standard PPO sweep

Optimal ranges and sensitivity rankings will be reported post-run with the same protocol as the main results: ≥ 20 *seeds, 95% bootstrap CI per cell.*

E.2.2 Ablation Study Details

Complete ablation study removing each component:

```
def detailed_ablation_study():
    """
    Systematic ablation of each InfoMARL component
    """
    configurations = {
        'Full InfoMARL': {
            'use_redundancy': True,
            'use_relevance': True,
            'use_capacity': True,
            'use_adaptive': True,
            'use_curriculum': True
        },
        'No Redundancy': {
            'use_redundancy': False,
            'use_relevance': True,
            'use_capacity': True,
            'use_adaptive': True,
            'use_curriculum': True
        },
        'No Relevance': {
            'use_redundancy': True,
            'use_relevance': False,
            'use_capacity': True,
            'use_adaptive': True,
            'use_curriculum': True
        },
        'No Capacity': {
            'use_redundancy': True,
            'use_relevance': True,
            'use_capacity': False,
            'use_adaptive': True,
```

```
            'use_curriculum': True
        },
        'No Adaptation': {
            'use_redundancy': True,
            'use_relevance': True,
            'use_capacity': True,
            'use_adaptive': False,
            'use_curriculum': True
        },
        'No Curriculum': {
            'use_redundancy': True,
            'use_relevance': True,
            'use_capacity': True,
            'use_adaptive': True,
            'use_curriculum': False
        }
    }

    metrics_to_track = [
        'final_performance',
        'convergence_speed',
        'sample_efficiency',
        'specialization_index',
        'robustness_score'
    ]

    results = {}

    for config_name, config in configurations.items():
        print(f"\nTesting: {config_name}")

        agent = train_with_ablation(config)

        results[config_name] = {
            'final_performance': evaluate_performance(agent),
            'convergence_speed': measure_convergence(agent),
            'sample_efficiency': measure_sample_efficiency(agent),
            'specialization_index': measure_specialization(agent),
            'robustness_score': measure_robustness(agent)
        }

    baseline = results['Full InfoMARL']
    component_importance = {}

    for config_name, metrics in results.items():
        if config_name != 'Full InfoMARL':
            degradation = 0
            for metric_name, value in metrics.items():
                baseline_value = baseline[metric_name]
                relative_change = (baseline_value - value) / baseline_value
                degradation += relative_change

            component = config_name.replace('No ', '')
            component_importance[component] = degradation / len(metrics_to_track)

    return results, component_importance
```

Ablation Results (table to be populated after runs).

Component Removed	Performance Drop	Convergence Delay	Specialization Loss
Conditional redundancy penalty	TBD	TBD	TBD
Relevance bonus	TBD	TBD	TBD
Capacity control	TBD	TBD	TBD
Adaptive scaling	TBD	TBD	TBD
Curriculum learning	TBD	TBD	TBD

The expected ordering, prior to running: the conditional-redundancy penalty should dominate the specialization-loss column; the capacity control should dominate the convergence-delay column; the relevance bonus should dominate the performance-drop column. We commit to this ordering in advance so that the run can falsify it.

E.2.3 Scaling Experiments

Testing how InfoMARL scales with number of agents:

```
def scaling_experiments():
    """
    Test scaling from 2 to 32 agents
    """
    agent_counts = [2, 4, 8, 16, 32]
    results = {}

    for n_agents in agent_counts:
        print(f"\nTesting with {n_agents} agents...")

        env = create_scaled_environment(n_agents)

        info_analysis = InformationTheoreticAnalysis(env)
        info_metrics = info_analysis.run_complete_analysis()

        start_time = time.time()
        agents = train_infomarl(env, n_agents)
        train_time = time.time() - start_time

        performance = evaluate_multi_agent(agents, env)

        results[n_agents] = {
            'H_joint': info_metrics['H_joint'],
            'H_conditional_avg': info_metrics['H_conditional_avg'],
            'total_parameters': sum(a.get_capacity() for a in agents),
            'training_time': train_time,
            'performance': performance,
            'efficiency': performance / (train_time * n_agents)
        }

    import scipy.stats

    ns = list(results.keys())
    H_joints = [results[n]['H_joint'] for n in ns]
    H_conds = [results[n]['H_conditional_avg'] for n in ns]

    log_n = np.log(ns)
    log_H_joint = np.log(H_joints)
    log_H_cond = np.log(H_conds)

    slope_joint, intercept_joint, r_joint, _, _ = scipy.stats.linregress(log_n, log_H_joint)
    slope_cond, intercept_cond, r_cond, _, _ = scipy.stats.linregress(log_n, log_H_cond)

    print(f"\nScaling laws:")
    print(f"H_joint ~ n^{slope_joint:.2f} (R^2={r_joint**2:.3f})")
    print(f"H_conditional ~ n^{slope_cond:.2f} (R^2={r_cond**2:.3f})")
```

```
    return results
```

Scaling protocol. The table below specifies the agent-count sweep; cells will be populated after runs.

# Agents	H_{joint} (bits)	$\bar{H}_{\text{cond}}$ (bits)	Total Parameters	Training Time
2	TBD	TBD	TBD	TBD
4	TBD	TBD	TBD	TBD
8	TBD	TBD	TBD	TBD
16	TBD	TBD	TBD	TBD

Hypothesized scaling. We expect H_{joint} to grow super-linearly in N (interactions add joint structure), $\bar{H}_{\text{cond}}$ to be approximately constant or slowly decreasing (each agent's marginal share of uncertainty shrinks as more teammates can carry information), and training time to grow at least linearly in N from per-agent gradient steps. Numerical exponents will be fitted from the run and reported with CIs.

E.3 Alternative Algorithms and Comparisons

E.3.1 InfoMARL vs. Communication-Based Methods

We compared InfoMARL against methods that use explicit communication:

```python
class CommunicationBaseline:
    """
    Baseline that uses communication instead of information regularization
    """

    def __init__(self, n_agents, comm_bandwidth):
        self.n_agents = n_agents
        self.comm_bandwidth = comm_bandwidth # bits per timestep
        self.comm_network = self._create_comm_network()

    def _create_comm_network(self):
        """Create network for generating messages"""
        return nn.Sequential(
            nn.Linear(self.obs_dim, 64),
            nn.ReLU(),
            nn.Linear(64, self.comm_bandwidth)
        )

    def communicate(self, observations):
        """Generate and broadcast messages"""
        messages = []
        for i, obs in enumerate(observations):
            message = torch.tanh(self.comm_network(obs))
            message_bits = (message > 0).float()
            messages.append(message_bits)

        return torch.stack(messages)

    def act(self, observations, messages):
        """Select actions using observations and messages"""
        enhanced_obs = []
        for i, obs in enumerate(observations):
            other_messages = torch.cat([
                messages[j] for j in range(self.n_agents) if j != i
            ])
            enhanced = torch.cat([obs, other_messages])
            enhanced_obs.append(enhanced)

        return [self.agents[i](enhanced_obs[i]) for i in range(self.n_agents)]
```

Comparison Results:

Method	Bandwidth	Performance	Parameters	Convergence
InfoMARL	0 bits	TBD	TBD	TBD
Comm-1bit	1 bit	TBD	TBD	TBD
Comm-4bit	4 bits	TBD	TBD	TBD
Comm-8bit	8 bits	TBD	TBD	TBD
Comm-Full	∞ bits	TBD	TBD	TBD

Expected pattern (P4): Comm-Full should be the upper bound; InfoMARL with $B = 0$ should approach it in scenarios where Δ_i is small, and fall well short in scenarios where Δ_i is large. The bandwidth sweep tests whether realized benefit tracks the predicted Δ_i.

E.3.2 Comparison with Graph Neural Network Approaches

```
class GNNMultiAgent:
    """
    Graph Neural Network approach for multi-agent coordination
    """

    def __init__(self, n_agents, hidden_dim=64):
        self.n_agents = n_agents

        self.gnn = GraphAttentionNetwork(
            node_features=self.obs_dim,
            edge_features=0,
            hidden_dim=hidden_dim,
            n_layers=3
        )

        self.policy_heads = nn.ModuleList([
            nn.Linear(hidden_dim, self.action_dim)
            for _ in range(n_agents)
        ])

    def forward(self, observations):
        edge_index = self._create_edge_index()

        node_features = torch.stack(observations)

        node_embeddings = self.gnn(node_features, edge_index)

        actions = []
        for i in range(self.n_agents):
            logits = self.policy_heads[i](node_embeddings[i])
            actions.append(torch.softmax(logits, dim=-1))

        return actions

    def _create_edge_index(self):
        edges = []
        for i in range(self.n_agents):
            for j in range(self.n_agents):
                if i != j:
                    edges.append([i, j])
        return torch.LongTensor(edges).T
```

Comparison with GNN (planned):

Metric	InfoMARL	GNN	GNN + Attention
Team reward	TBD	TBD	TBD
Parameters	TBD	TBD	TBD
Training wall-clock	TBD	TBD	TBD
Specialization index $I(\hat{Z}_i; \hat{Z}_j \mid R)$	TBD	TBD	TBD

Hypothesis. GNN approaches implicitly broadcast information through message passing, which we expect to *reduce* the conditional-MI signal we are tracking—a feature, not a bug, for the GNN, but it complicates direct specialization comparison. We will report whichever direction the data shows.

E.4 Theoretical Extensions

E.4.1 Extension to Partial Observability

The framework extends to POMDPs by considering belief states:

Definition E.1 (Belief-Conditional Entropy): For POMDP with belief state b_i:

$$H(\mathcal{Z}_i|b_i, \mathcal{Z}_{-i}) = \mathbb{E}_{b_i}[H(\mathcal{Z}_i|s, \mathcal{Z}_{-i})]$$

Theorem E.4 (POMDP Capacity): In POMDPs, required capacity increases by the belief entropy:

$$C^*_{\text{POMDP}} = H(\mathcal{Z}_i|\mathcal{Z}_{-i}) + \beta \cdot H(s|o_i)$$

where $\beta \in [0, 1]$ depends on the mixing time of the belief MDP.

E.4.2 Extension to Continuous Time

For continuous-time multi-agent systems:

Definition E.2 (Information Rate in Continuous Time):

$$\mathcal{R}_{\pi_i}(t) = \lim_{\Delta t \to 0} \frac{I(\mathcal{Z}_i(t); \pi_i(t))}{\Delta t}$$

Theorem E.5 (Continuous-Time Capacity): The minimum capacity for continuous-time coordination is:

$$C^*_{\text{continuous}} = \int_0^T \mathcal{R}_{\mathcal{Z}_i|\mathcal{Z}_{-i}}(t)dt$$

where $\mathcal{R}_{\mathcal{Z}_i|\mathcal{Z}_{-i}}(t)$ is the conditional information rate.

E.5 Implementation Optimizations

E.5.1 GPU-Accelerated Entropy Estimation

```
import torch
import torch.nn.functional as F

class GPUEntropyEstimator:
    """
    GPU-accelerated entropy estimation for large-scale systems
    """

    @staticmethod
    @torch.jit.script
    def estimate_entropy_discrete_gpu(samples: torch.Tensor) -> float:
        """
```

```
        Fast GPU entropy estimation
        """
        unique, counts = torch.unique(samples, return_counts=True)

        probs = counts.float() / len(samples)

        entropy = -torch.sum(probs * torch.log2(probs + 1e-10))

        k = len(unique)
        n = len(samples)
        correction = (k - 1) / (2 * n)

        return (entropy + correction).item()

    @staticmethod
    def batch_conditional_entropy_gpu(
        x_samples: torch.Tensor,
        y_samples: torch.Tensor,
        batch_size: int = 10000
    ) -> float:
        """
        Batch computation of conditional entropy on GPU
        """
        device = x_samples.device
        n_samples = len(x_samples)

        H_conditional = 0.0

        for i in range(0, n_samples, batch_size):
            batch_x = x_samples[i:i+batch_size]
            batch_y = y_samples[i:i+batch_size]

            joint = torch.cat([batch_x, batch_y], dim=1)
            H_joint = GPUEntropyEstimator.estimate_entropy_discrete_gpu(joint)

            H_y = GPUEntropyEstimator.estimate_entropy_discrete_gpu(batch_y)

            batch_weight = len(batch_x) / n_samples
            H_conditional += batch_weight * (H_joint - H_y)

        return H_conditional
```

E.5.2 Distributed Training Optimizations

```
import torch.distributed as dist
from torch.nn.parallel import DistributedDataParallel as DDP

class DistributedInfoMARL:
    """
    Distributed implementation for multi-GPU training
    """

    def __init__(self, rank, world_size):
        self.rank = rank
        self.world_size = world_size

        dist.init_process_group(backend='nccl', rank=rank, world_size=world_size)

        self.local_agents = self._create_local_agents()

    def _create_local_agents(self):
        agents_per_rank = self.n_agents // self.world_size
        start_idx = self.rank * agents_per_rank
        end_idx = start_idx + agents_per_rank

        local_agents = []
```

```
        for i in range(start_idx, end_idx):
            agent = create_agent(i)
            agent = DDP(agent, device_ids=[self.rank])
            local_agents.append(agent)

        return local_agents

    def distributed_train_step(self, batch):
        local_losses = []
        for agent in self.local_agents:
            loss = agent.compute_loss(batch)
            local_losses.append(loss)

        total_loss = sum(local_losses)

        total_loss.backward()

        for agent in self.local_agents:
            agent.optimizer.step()
            agent.optimizer.zero_grad()

        if self.step % 100 == 0:
            self.sync_information_estimates()

    def sync_information_estimates(self):
        local_mi = torch.tensor(self.local_mi_estimates)

        dist.all_reduce(local_mi, op=dist.ReduceOp.AVG)

        self.global_mi_estimates = local_mi.numpy()
```

E.6 Reproducibility Details

E.6.1 Random Seeds and Determinism

```
def set_reproducible_seeds(seed: int = 42):
    """
    Set all random seeds for reproducibility
    """
    import random
    import numpy as np
    import torch

    random.seed(seed)

    np.random.seed(seed)

    torch.manual_seed(seed)
    torch.cuda.manual_seed(seed)
    torch.cuda.manual_seed_all(seed)

    torch.backends.cudnn.deterministic = True
    torch.backends.cudnn.benchmark = False

    os.environ['PYTHONHASHSEED'] = str(seed)

    print(f"All seeds set to {seed} for reproducibility")
```

E.6.2 Computational Resources (planned)

Bucket Brigade is intentionally small (N=4 agents, $|\mathcal{A}_i|$=4, discrete observations), so the protocol of Section 7 is intended to be runnable on a single workstation-class GPU. We will report the exact hardware used and per-experiment wall-clock times alongside the run, in the form of the table below.

Resource	Specification
GPU(s)	TBD
CPU	TBD
RAM	TBD
OS / CUDA / PyTorch	TBD

E.6.3 Training Times (template)

Experiment	Training Time	Evaluation Time	Total Time
Capacity sweep	TBD	TBD	TBD
Sample complexity	TBD	TBD	TBD
Specialization	TBD	TBD	TBD
Communication	TBD	TBD	TBD
Full protocol	TBD	TBD	TBD

E.7 Code Availability

The complete codebase is organized as follows:

```
infomarl/
|-- core/
|   |-- algorithms.py      # InfoMARL implementation
|   |-- estimators.py      # Entropy/MI estimators
|   `-- networks.py        # Neural network architectures
|-- envs/
|   |-- bucket_brigade.py  # Bucket Brigade environment
|   `-- wrappers.py        # Environment wrappers
|-- experiments/
|   |-- capacity.py        # Capacity experiments
|   |-- complexity.py      # Sample complexity experiments
|   `-- ablation.py        # Ablation studies
|-- analysis/
|   |-- information.py     # Information-theoretic analysis
|   `-- visualization.py   # Plotting utilities
|-- configs/
|   `-- default.yaml       # Default hyperparameters
`-- scripts/
    |-- train.py           # Training script
    |-- evaluate.py        # Evaluation script
    `-- reproduce.sh       # Reproduction script
```

To reproduce all experiments:

```
./scripts/reproduce.sh --seed 42 --gpu 0
```

E.8 Summary

This appendix collected:

1. Extended arguments for special cases (linear–Gaussian; the v1 phase-transition sketch, retained as heuristic).
2. Templates for the planned experiments (sensitivity, ablation, scaling) that will be populated when the runs of Section 7 complete.

3. Comparison structures against communication-based and GNN-based methods.
4. Theoretical extensions to POMDPs and continuous time, stated as conjectures.
5. Implementation sketches for GPU and distributed training, against the day we run them.
6. Reproducibility commitments: seeds, planned hardware reporting, and the eventual code-release structure.

Preprint v2; target venue under consideration.